

Ontwerp en realisatie van een performante provisioner voor een virtuele Windows-en Linux omgevingen in Rust.

Jens Gosseye.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. B. Van Vreckem

Co-promotor: Dhr. J. Courtens

Academiejaar: 2025–2026

Tweede examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Curabitur nunc magna, posuere eget, venenatis eu, vehicula ac, velit. Aenean ornare, massa a accumsan pulvinar, quam lorem laoreet purus, eu sodales magna risus molestie lorem. Nunc erat velit, hendrerit quis, malesuada ut, aliquam vitae, wisi. Sed posuere. Suspendisse ipsum arcu, scelerisque nec, aliquam eu, molestie tincidunt, justo. Phasellus iaculis. Sed posuere lorem non ipsum. Pellentesque dapibus. Suspendisse quam libero, laoreet a, tincidunt eget, consequat at, est. Nullam ut lectus non enim consequat facilisis. Mauris leo. Quisque pede ligula, auctor vel, pellentesque vel, posuere id, turpis. Cras ipsum sem, cursus et, facilisis ut, tempus euismod, quam. Suspendisse tristique dolor eu orci. Mauris mattis. Aenean semper. Vivamus tortor magna, facilisis id, varius mattis, hendrerit in, justo. Integer purus.

Vivamus adipiscing. Curabitur imperdiet tempus turpis. Vivamus sapien dolor, congue venenatis, euismod eget, porta rhoncus, magna. Proin condimentum pretium enim. Fusce fringilla, libero et venenatis facilisis, eros enim cursus arcu, vitae facilisis odio augue vitae orci. Aliquam varius nibh ut odio. Sed condimentum condimentum nunc. Pellentesque eget massa. Pellentesque quis mauris. Donec ut ligula ac pede pulvinar lobortis. Pellentesque euismod. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent elit. Ut laoreet ornare est. Phasellus gravida vulputate nulla. Donec sit amet arcu ut sem tempor malesuada. Praesent hendrerit augue in urna. Proin enim ante, ornare vel, consequat ut, blandit in, justo. Donec felis elit, dignissim sed, sagittis ut, ullamcorper a, nulla. Aenean pharetra vulputate odio.

Samenvatting

Deze bachelorproef behandelt het ontwerp en de realisatie van RustProv, een provisioner voor virtuele Windows- en Linuxomgevingen. Binnen de opleiding Toegepaste Informatica aan HOGENT wordt voor het opzetten van labo- en ontwikkelomgevingen vaak gebruikgemaakt van Vagrant, maar in de praktijk leidt dit soms tot trage opstarttijden, foutgevoelige provisioning en beperkte controle over het proces. Vanuit dit punt werd onderzocht hoe een eigen provisioner in Rust ontwikkeld kan worden die beter aansluit bij de behoeften van een onderwijsomgeving. Het doel van deze bachelorproef is een proof of concept te realiseren dat virtuele machines automatisch kan aanmaken, configureren en provisionen op een consistente en betrouwbare manier. Daarbij wordt ook gekeken naar praktische verbeteringen, zoals het direct koppelen van een VDI-bestand, het gebruik van shared folders en het ondersteunen van zowel één VM als meerdere VM's. De performantie van RustProv wordt bovendien vergeleken met die van Vagrant om na te gaan of de nieuwe aanpak sneller of efficiënter is. Deze bachelorproef levert daarmee een werkende basis op voor verdere uitbreiding van geautomatiseerde VM-provisioning in een onderwijscontext.

Tweede zittijd

Inhoudsopgave

Lijst van figuren	x
Lijst van tabellen	xi
Lijst van codefragmenten	xii
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	1
1.3 Onderzoeksdoelstelling	2
1.4 Opzet van deze bachelorproef	2
2 Stand van zaken	4
2.1 Provisioner en provisioning	4
2.2 Infrastructure as Code	5
2.3 Rust als programmeertaal	6
2.4 Educatieve context	7
2.5 RustProv	7
2.6 Samenvatting	8
3 Methodologie	9
3.1 Aanpak	9
3.2 Fase 1: Literatuurstudie	9
3.2.1 Doelstelling	9
3.2.2 Deliverables	10
3.2.3 Onderzoeksmethoden	10
3.2.4 Verantwoording	10
3.3 Fase 2: Requirements-analyse	10
3.3.1 Doelstelling	10
3.3.2 Deliverables	10
3.3.3 Onderzoeksmethoden	10
3.3.4 Verantwoording	11
3.4 Fase 3: Proof of concept en sprintaanpak	11
3.4.1 Doelstelling	11
3.4.2 Deliverables	11
3.4.3 Onderzoeksmethoden	11
3.4.4 Verantwoording	11

3.5	Fase 4: Testopstelling en evaluatie	11
3.5.1	Doelstelling	11
3.5.2	Deliverables	12
3.5.3	Onderzoeksmethoden	12
3.5.4	Verantwoording	12
3.6	Fase 5: Analyse van de resultaten	12
3.6.1	Doelstelling	12
3.6.2	Deliverables	12
3.6.3	Onderzoeksmethoden	12
3.6.4	Verantwoording	13
3.7	Samenvatting	13
4	Iteratieve uitwerking	14
4.1	Fase 1: Rust aanleren	14
4.1.1	Week 1-2	14
4.2	Fase 2: Sprint 1 - opstarten	14
4.2.1	Week 3	15
4.2.2	Week 4	15
4.3	Fase 3: Sprint 2 - provisionen	15
4.3.1	Week 5	15
4.3.2	Week 6	15
4.4	Fase 4: Sprint 3 - configuratie	15
4.4.1	Week 7	15
4.4.2	Week 8	16
4.5	Fase 5: Sprint 4 - functies	16
4.5.1	Week 9	16
4.5.2	Week 10	16
4.6	Fase 6: Sprint 5 - foutafhandeling	16
4.6.1	Week 11	16
4.6.2	Week 12	17
4.7	Planning	17
5	Requirements-analyse	18
5.1	Minimale vereisten	18
5.2	Vereisten voor RustProv	19
5.3	Functionele en niet-functionele vereisten	20
5.4	Afbakening	21
6	Toolselectie	22
6.1	VirtualBox	22
6.2	Rust	22
6.3	ssh2	23
6.4	TOML en Serde	23

6.5	Clap	23
6.6	IndexMap	24
6.7	Crossterm	24
6.8	PSTools	24
7	Ontwerp	25
7.1	Basisfunctionaliteit	25
7.2	Gebruiksgemak en beheer	25
7.3	Herstel- en noodfunctionaliteit	26
7.3.1	Provisioningworkflow	26
7.4	SSH-verbinding	26
7.5	Linux- en Windowsprovisioning	26
7.5.1	Linuxprovisioning	27
7.5.2	Windowsprovisioning	27
7.6	Uitbreidingen	27
7.6.1	Snapshots	27
7.6.2	Priority	27
7.6.3	Stop, remove en restore	27
7.6.4	Kill	28
7.7	Ondersteuning voor Linux en Windows	28
8	Implementatie	30
8.1	Sprint 1: opstarten van virtuele machines	30
8.2	Sprint 2: provisioning via SSH	31
8.3	Sprint 3: externe configuratie	31
8.4	Sprint 4: uitbreidingen en gebruiksgemak	32
8.5	Sprint 5: foutafhandeling en robuustheid	32
9	Experimentopzet	34
9.1	Windows-omgeving	35
9.1.1	Eén virtuele machine	35
9.1.2	Meerdere virtuele machines	35
9.2	Linux-omgeving	36
9.2.1	Eén virtuele machine	36
9.2.2	Meerdere virtuele machines	37
10	Resultaten	52
10.1	Meetmethode	52
10.1.1	PowerShell-meetscripts	53
10.2	Meting van één Linux-VM	53
10.3	Meting van meerdere Linux-VM's	54
10.4	Meting van één Windows-VM	54
10.5	Meting van meerdere Windows-VM's	55

10.6	Vergelijking tussen RustProv en Vagrant	55
11	Conclusie	56
A	Onderzoeksvoorstel	58
A.1	Inleiding	58
A.2	Literatuurstudie	59
A.2.1	Infrastructure as Code	59
A.2.2	Educatieve context	59
A.2.3	Rust als programmeertaal	60
A.3	Methodologie	60
A.3.1	Fase 1: Rust aanleren	60
A.3.2	Fase 2: Sprint 1 - opstarten	60
A.3.3	Fase 3: Sprint 2 - provisionen	60
A.3.4	Fase 4: Sprint 3 - configuratie	61
A.3.5	Fase 5: Sprint 4 - functies	61
A.3.6	Fase 6: Sprint 5 - foutafhandeling	61
A.3.7	Planning	61
A.4	Verwacht resultaat, conclusie	61

Lijst van figuren

A.1 Gantt diagram met de verschillende fasen en milestones van het onderzoek.	60
---	----

Lijst van tabellen

10.1 Meting van één Linux-VM met RustProv en Vagrant.	54
10.2 Totale uitvoeringstijd van meerdere Linux-VM's met RustProv en Vagrant.	54
10.3 Meting van één Windows-VM met RustProv en Vagrant.	55
10.4 Totale uitvoeringstijd van meerdere Windows-VM's met RustProv en Vagrant.	55

Lijst van codefragmenten

7.1	Voorbeeld van platformafhankelijke procesafsluiting.	28
7.2	Voorbeeld van een TOML-configuratie voor een virtuele machine.	29
8.1	Voorbeeld van console-uitvoer na sprint 1.	31
8.2	Voorbeeld van console-uitvoer bij Linux-provisioning na sprint 2.	31
8.3	Voorbeeld van console-uitvoer bij Windows-provisioning na sprint 2.	31
8.4	Voorbeeld van console-uitvoer na het inlezen van TOML-configuratie.	32
8.5	Voorbeeld van console-uitvoer voor snapshots en statuscontrole na sprint 4.	32
8.6	Voorbeeld van console-uitvoer na sprint 5 met meerdere VM's en prioriteit.	33
8.7	Voorbeeld van console-uitvoer bij foutafhandeling.	33
9.1	Provisioningscript voor de Windows-client.	35
9.2	Provisioningscript voor Windows Server 1.	35
9.3	Provisioningscript voor Windows Server 2.	35
9.4	Provisioningscript voor de Windows-client in de meerledige opstelling.	36
9.5	Eenvoudig provisioningscript voor één Linux-VM.	36
9.6	RustProv-configuratie voor één Linux-VM.	37
9.7	Vagrant-configuratie voor één Linux-VM	37
9.8	Linux-script 1: installatie van database en Loki.	39
9.9	Linux-script 2: installatie van WordPress en Promtail.	42
9.10	Linux-script 3: installatie van Grafana en monitoring.	47
9.11	RustProv-configuratie voor meerdere Linux-VM's.	50
9.12	Vagrant-configuratie voor meerdere Linux-VM's.	51
10.1	PowerShell-script voor het meten van één virtuele machine.	53
10.2	PowerShell-script voor het meten van meerdere virtuele machines.	53

1

Inleiding

Binnen de richting IT Infrastructure Engineer aan HOGENT wordt gebruikgemaakt van verschillende provisioningtools om virtuele Windows- en Linuxomgevingen op te zetten. Een veelgebruikte tool is Vagrant: studenten gebruiken deze om ontwikkel- en labo-omgevingen te automatiseren op basis van een declaratieve configuratiefile. In de praktijk treden echter verschillende problemen op tijdens dit proces: virtuele machines starten traag op, het provisioningproces loopt soms vast of wordt niet gestart door time-outs, virtuele machines kunnen onverwachte *kernel panics* vertonen en bij het onderbreken van het provisioningproces blijft dit soms op de achtergrond verder draaien, waardoor de gebruiker het proces manueel moet stopzetten. Dit zorgt voor veel frustraties en tijdverlies, zowel in onderwijs- als bedrijfscontext.

1.1. Probleemstelling

De huidige Vagrant-gebaseerde aanpak voor provisioning in de opleiding Toegepaste Informatica aan HOGENT leidt regelmatig tot trage opstarttijden, vastlopende provisioning en instabiele virtuele machines. Dit zorgt ervoor dat studenten en lectoren tijd verliezen tijdens de lessen, labo's en evaluaties. Hierdoor wordt de focus van de lessen verlegd van het leren naar het oplossen van verschillende technische problemen. De doelgroep van dit onderzoek bestaat uit studenten en docenten binnen de richting IT Infrastructure Engineer aan HOGENT. Zij hebben nood aan een meer performante en betrouwbare provisioningoplossing die beter is afgestemd op een onderwijs- en laboomgeving.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag van deze bachelorproef luidt:

Hoe ontwerp en realiseer je een performante provisioner voor virtuele Windows- en Linuxomgevingen, die beter inspeelt op de behoeften van het onderwijs dan de huidige Vagrant-gebaseerde aanpak in Rust?

Om deze hoofdonderzoeksvraag te concretiseren, worden volgende deelvragen onderzocht:

- Welke functionaliteiten zijn essentieel voor een provisioner die in onderwijs- en labo-omgevingen wordt gebruikt?
- Op welke besturingssystemen zal de provisioner zelf draaien?
- Voor welke gast-besturingssystemen moet de provisioner virtuele machines kunnen opbouwen en configureren?

1.3. Onderzoeksdoelstelling

Het doel van deze bachelorproef is een proof of concept provisioner te schrijven in Rust. Deze zal virtuele Linux- en Windows-omgevingen kunnen opzetten op een betrouwbare manier. De succescriteria omvatten een werkende command-line tool die op Windows kan worden uitgevoerd, en ondersteuning voor kernfunctionaliteiten zoals het opstarten van de VM, provisioning via SSH en configuratie via een declaratief bestand. Daarnaast wordt een aantoonbare verbetering in de provisioningtijd en stabiliteit tegenover de bestaande Vagrant-omgevingen verwacht. Tot slot zal er documentatie en een gebruikershandleiding voorzien worden, zodat de tool inzetbaar is binnen de opleiding Toegepaste Informatica aan HOGENT.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 4 wordt het verloop van de ontwikkeling van RustProv beschreven aan de hand van de verschillende sprints.

In Hoofdstuk 7 wordt het ontwerp van de provisioner toegelicht, met aandacht voor de basisfunctionaliteit, de provisioningworkflow en de platformafhankelijke uitvoering.

In Hoofdstuk 8 wordt besproken hoe RustProv stapsgewijs werd opgebouwd en welke functionaliteiten per sprint werden toegevoegd.

In Hoofdstuk 9 wordt de opzet van de experimenten toegelicht, inclusief de test-scenario's voor Windows en Linux.

In Hoofdstuk 10 worden de meetresultaten en de vergelijking met Vagrant besproken.

In Hoofdstuk 11, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

Dit hoofdstuk beschrijft de stand van zaken binnen het onderzoeksdomein van virtuele machine provisioning. Eerst wordt toegelicht wat provisioning inhoudt en welke rol een provisioner speelt bij het automatisch opbouwen en configureren van IT-omgevingen. Vervolgens wordt Infrastructure as Code besproken als bredere methodologie waarin configuratiebestanden en automatisering worden gebruikt. Daarna wordt Rust behandeld als programmeertaal voor systeemgerichte software. Vervolgens wordt de educatieve context van virtuele machines besproken. Tot slot wordt RustProv geïntroduceerd als de concrete toepassing van deze concepten binnen deze bachelorproef.

Deze opbouw gaat van algemene begrippen naar de specifieke oplossing die in dit onderzoek wordt ontwikkeld. Hierdoor beschikt de lezer eerst over de nodige achtergrond voordat de werking en het ontwerp van RustProv worden toegelicht.

2.1. Provisioner en provisioning

Provisioning verwijst naar het voorbereiden en beschikbaar maken van IT-infrastructuur. Dit kan onder meer betrekking hebben op servers, virtuele machines, netwerken, opslag en toegangsrechten. Bij geautomatiseerde provisioning worden deze stappen uitgevoerd door een tool, configuratiebestand of script in plaats van door een gebruiker die elke stap handmatig uitvoert. Hierdoor kan een omgeving sneller en consistentter worden opgebouwd (**RedHat2023**).

Een provisioner is een toepassing die infrastructuur automatisch kan aanmaken en voorbereiden voor gebruik. In het geval van virtuele machines omvat dit bijvoorbeeld het aanmaken of herkennen van een VM, het instellen van hardware- en netwerkparameters, het starten van de machine en het uitvoeren van configuratiestappen op het gastbesturingssysteem (**RedHat2023; HashiCorp2025**). Een provisioner beperkt zich dus niet tot het beheren van de levenscyclus van een VM, maar

zorgt er ook voor dat de machine in een bruikbare toestand wordt achtergelaten. Een belangrijk kenmerk van geautomatiseerde provisioning is herhaalbaarheid (**IBM2025**). Wanneer dezelfde configuratie meerdere keren wordt uitgevoerd, moet dit in principe leiden tot een gelijkaardige omgeving. Dit is relevant in situaties waarin dezelfde omgeving door verschillende gebruikers of op verschillende momenten opnieuw moet worden opgebouwd. Een externe configuratie en geautomatiseerde scripts helpen om verschillen tussen manuele installaties te beperken. Provisioning bestaat doorgaans uit meerdere opeenvolgende stappen. Eerst wordt de infrastructuur aangemaakt of geselecteerd. Vervolgens worden de nodige hardware- en netwerkparameters ingesteld. Daarna wordt de machine gestart en bereikbaar gemaakt, waarna configuratiescripts of andere installatiestappen op het gastbesturingssysteem kunnen worden uitgevoerd. Vagrant is een voorbeeld van een bestaande tool die virtuele ontwikkel- en labomgevingen kan automatiseren. Vagrant ondersteunt provisioners waarmee software kan worden geïnstalleerd, configuraties kunnen worden aangepast en scripts op een gastmachine kunnen worden uitgevoerd (**HashiCorp2025**). Provisioning kan onder andere plaatsvinden tijdens `vagrant up`, via `vagrant provision` of wanneer een expliciete provisioningsoptie wordt meegegeven (**HashiCorp2025**). Hoewel een provisioner verschillende taken kan automatiseren, blijft de uiteindelijke werking afhankelijk van de virtualisatieomgeving, de netwerkconfiguratie en het gastbesturingssysteem. Daarom moet een provisioner niet alleen rekening houden met de gewenste configuratie, maar ook met de manier waarop VM's bereikbaar worden gemaakt en scripts op verschillende platformen worden uitgevoerd.

2.2. Infrastructure as Code

Infrastructure as Code, afgekort als IaC, is een aanpak waarbij infrastructuur wordt beschreven en beheerd door middel van configuratiebestanden en programmatische definities (**IBM2025**). In plaats van infrastructuur volledig handmatig te configureren, wordt de gewenste toestand beschreven in code of in een declaratief configuratiebestand. Deze configuratie kan daarna automatisch worden toegepast (**IBM2025**).

Het gebruik van IaC heeft verschillende gevolgen voor de manier waarop infrastructuur wordt beheerd. Configuraties kunnen worden bijgehouden, hergebruikt en opnieuw uitgevoerd. Daardoor wordt het eenvoudiger om wijzigingen te controleren en om een omgeving opnieuw op te bouwen wanneer dat nodig is (**IBM2025**). De configuratie vormt bovendien een expliciete beschrijving van de gewenste toestand van de omgeving (**IBM2025**).

Binnen IaC kan een onderscheid worden gemaakt tussen verschillende soorten tools. Provisioningtools richten zich op het creëren en beschikbaar maken van infrastructuur. Configuratiemanagementtools richten zich eerder op het installeren

en configureren van software op bestaande systemen. Orchestrationtools bepalen daarnaast vaak de volgorde en samenwerking tussen verschillende systemen of taken.

Een configuratiebestand kan zowel infrastructuurparameters als configuratiestappen beschrijven. Zo kunnen bijvoorbeeld de naam van een VM, de hoeveelheid geheugen, het aantal processorkernen, de netwerkconfiguratie en de uit te voeren scripts in één configuratie worden vastgelegd. Hierdoor wordt de relatie tussen de gewenste omgeving en de uitvoering ervan explicieter.

Voor dit onderzoek is vooral het provisioninggedeelte van IaC relevant. De te ontwikkelen tool gebruikt een externe configuratie om VM's te beschrijven en de provisioningworkflow aan te sturen. De configuratie vormt daarmee de invoer voor het automatisch aanmaken, starten en provisionen van de virtuele machines.

2.3. Rust als programmeertaal

Rust is een programmeertaal die gericht is op het ontwikkelen van betrouwbare en efficiënte software. Een belangrijk onderdeel van Rust is het ownershipmodel. Dit model, samen met borrowing en lifetimes, laat de compiler geheugenveiligheid controleren tijdens het compileren. Volgens de officiële Rust-documentatie kan Rust hierdoor geheugenveiligheid garanderen zonder gebruik te maken van een garbage collector (**Rust2025**).

Het ownershipmodel bepaalt welke variabele eigenaar is van een waarde en wanneer die waarde automatisch wordt vrijgegeven. Borrowing maakt het mogelijk om tijdelijk naar data te verwijzen zonder het eigenaarschap over te dragen. Deze regels worden door de compiler gecontroleerd voordat het programma wordt uitgevoerd (**Rust2025**).

Rust is relevant voor het ontwikkelen van systeemgerichte software omdat programma's vaak rechtstreeks moeten samenwerken met processen, bestanden, netwerkverbindingen en het onderliggende besturingssysteem. Daarbij is niet alleen functionaliteit belangrijk, maar ook voorspelbaar gedrag wanneer een externe actie mislukt.

Voor de te ontwikkelen provisioner is het bovendien belangrijk dat de toepassing op verschillende hostbesturingssystemen kan worden gecompileerd en uitgevoerd. Platformafhankelijke onderdelen, zoals het beëindigen van processen of het bepalen van de gebruikersmap, kunnen daarbij afzonderlijk per besturingssysteem worden geïmplementeerd.

De provisioner gebruikt daarnaast een virtualisatieplatform met een commandline-interface om virtuele machines te beheren. VirtualBox biedt hiervoor het programma VBoxManage. Met deze interface kunnen VM's vanuit het hostbesturingssysteem via commando's worden aangemaakt, geconfigureerd, gestart en beheerd (**Oracle2022**).

2.4. Educatieve context

Virtuele machines worden in onderwijsomgevingen gebruikt om studenten een gecontroleerde en geïsoleerde technische omgeving aan te bieden. Daardoor kunnen studenten experimenteren met besturingssystemen, netwerken en servertoepassingen zonder rechtstreeks wijzigingen aan te brengen aan de fysieke hostomgeving.

Een uitdaging binnen deze context is dat de beschikbare hardware en tijd vaak beperkt zijn. Onderzoek naar het gebruik van virtuele omgevingen voor onderwijs toont aan dat onder meer performantie, opslag, netwerkcommunicatie en de keuze van virtualisatieplatformen een rol spelen bij het opbouwen van bruikbare labo-omgevingen (**Robison2012**).

Een provisioningtool kan in deze context helpen om een vaste beginsituatie voor studenten te creëren. Wanneer de VM's en scripts automatisch worden opgebouwd, moeten studenten minder tijd besteden aan handmatige installatie en configuratie. De beschikbare lestijd kan daardoor meer gericht worden op de inhoud van de oefening.

Voor een provisioner binnen een onderwijscontext zijn daarom niet alleen technische mogelijkheden belangrijk. De tool moet ook eenvoudig configureerbaar zijn, duidelijke feedback geven en kunnen omgaan met scenario's met één of meerdere virtuele machines. Herhaalbaarheid, voorspelbaarheid en beperkte manuele tussenkomst zijn hierbij belangrijke eigenschappen.

2.5. RustProv

RustProv is de provisioner die binnen deze bachelorproef wordt ontworpen en gerealiseerd. De tool combineert de eerder beschreven concepten van provisioning, Infrastructure as Code en systeemgerichte softwareontwikkeling. RustProv richt zich op het automatisch opbouwen en provisionen van virtuele Windows- en Linuxomgevingen met VirtualBox als virtualisatieplatform.

De gebruiker beschrijft de gewenste VM-opstelling in een extern TOML-configuratiebestand. Daarin kunnen onder andere de naam van de VM, de gebruikte image, het aantal processorkernen, de hoeveelheid geheugen, de uit te voeren scripts en optionele netwerk- of prioriteitsinstellingen worden vastgelegd. Hierdoor blijft de configuratie buiten de broncode en kunnen verschillende testopstellingen worden gebruikt zonder het programma opnieuw te compileren.

Na het inlezen van de configuratie maakt RustProv de virtuele machines aan of herkent het machines die al bestaan. Vervolgens worden de VM-instellingen aangepast en worden de virtuele schijven gekoppeld. Via VBoxManage kunnen deze beheeracties vanuit de commandline worden uitgevoerd (**Oracle2022**).

Wanneer meerdere VM's worden gebruikt, kan RustProv de machines eerst opstarten en daarna de provisioning uitvoeren. De configuratie kan ook een prioriteit be-

vatten, zodat VM's in een vooraf bepaalde volgorde worden verwerkt. Dit is relevant wanneer een scenario uit verschillende systemen bestaat die onderling afhankelijk zijn.

Na het opstarten wordt netwerktoegang voorzien via port forwarding. RustProv gebruikt SSH om verbinding te maken met de gastmachine en om provisioning-opdrachten uit te voeren. Voor Linux worden scripts via een shellomgeving uitgevoerd. Voor Windows wordt een afzonderlijk uitvoeringspad gebruikt omdat de scriptuitvoering en het rechtenbeheer van beide platformen van elkaar verschillen. RustProv ondersteunt daarnaast functies voor statuscontrole, snapshots, herstellen, stoppen, verwijderen en foutafhandeling. Deze functies zorgen ervoor dat de tool niet alleen een VM kan opstarten, maar ook de levenscyclus van de omgeving kan beheren. Hierdoor sluit de functionaliteit aan bij de eerder beschreven kenmerken van een provisioner.

2.6. Samenvatting

Provisioning bestaat uit het automatisch opbouwen en voorbereiden van IT-infrastructuur (**RedHat2023**). Infrastructure as Code biedt een manier om de gewenste toestand van deze infrastructuur in configuratiebestanden vast te leggen en herhaalbaar toe te passen (**IBM2025**). Rust biedt een geschikte basis voor de ontwikkeling van systeemgerichte software dankzij het ownershipmodel en de compile-time controle van geheugenveiligheid (**Rust2025**).

Binnen deze bachelorproef worden deze concepten toegepast in RustProv. De tool combineert VirtualBox-beheer, externe TOML-configuratie, SSH-gebaseerde scriptuitvoering en ondersteuning voor virtuele Windows- en Linuxomgevingen. De volgende hoofdstukken bouwen hierop voort door eerst de requirements en de gekozen tools te bespreken en daarna het ontwerp en de implementatie van RustProv toe te lichten.

3

Methodologie

In dit hoofdstuk wordt de opzet van het onderzoek en de ontwikkeling van Rust-Prov op hoofdlijnen beschreven. Het traject werd opgevat als een iteratief proces waarin eerst de theoretische basis werd onderzocht, vervolgens de functionele vereisten werden vastgelegd en daarna stapsgewijs een proof of concept werd uitgewerkt, getest en verfijnd. Deze gefaseerde aanpak maakte het mogelijk om de scope beheersbaar te houden en om elke tussenstap afzonderlijk te evalueren.

3.1. Aanpak

De gekozen werkwijze bestond uit opeenvolgende fasen die telkens een duidelijk doel hadden. Eerst werd de relevante kennis over provisioning, virtualisatie en Rust opgebouwd via literatuurstudie. Daarna werden de functionele en niet-functionele vereisten geanalyseerd. Op basis daarvan werd een proof of concept gerealiseerd, waarna de implementatie in meerdere sprints werd uitgebreid en getest. Deze opbouw sloot aan bij de beperkte projectduur en liet toe om vroeg in het traject al werkende tussenresultaten te bekomen.

3.2. Fase 1: Literatuurstudie

3.2.1. Doelstelling

De eerste fase had als doel een inhoudelijke en technische basis te leggen voor het project. Daarbij werd onderzocht wat provisioning precies inhoudt, welke rol virtuele machines spelen in een educatieve context en waarom Rust een geschikte programmeertaal is voor dit type tool.

3.2.2. Deliverables

De deliverables van deze fase waren een eerste inhoudelijk kader voor de bachelorproef en een afbakening van het onderzoeksdomein. Daarnaast werd duidelijk welke bestaande oplossingen, zoals Vagrant, relevant zijn als referentiepunt voor de verdere uitwerking van RustProv.

3.2.3. Onderzoeksmethoden

Voor deze fase werd voornamelijk literatuuronderzoek toegepast. Daarbij werd gebruikgemaakt van wetenschappelijke bronnen, documentatie van gebruikte technologieën en achtergrondinformatie over Infrastructure as Code, virtualisatie en provisioning. Deze methode was geschikt omdat de vraagstukken in deze fase vooral conceptueel en vergelijkend van aard waren.

3.2.4. Verantwoording

Een literatuurstudie was noodzakelijk om de probleemstelling correct te positioneren en om de latere requirements-analyse niet louter intuïtief op te bouwen. Door eerst de bestaande kennis te bestuderen, kon de implementatie beter worden afgestemd op de verwachtingen van een onderwijscontext en op de technische beperkingen van virtuele omgevingen.

3.3. Fase 2: Requirements-analyse

3.3.1. Doelstelling

In de tweede fase werd bepaald welke functies RustProv minimaal moest ondersteunen om als volwaardige provisioner te kunnen functioneren. Daarbij werd ook gekeken naar de extra vereisten die specifiek relevant zijn voor Windows- en Linuxomgevingen en voor het werken met één of meerdere VM's.

3.3.2. Deliverables

Het resultaat van deze fase was een requirements-overzicht met functionele en niet-functionele vereisten. Daarin kwamen onder meer configuratie via TOML, SSH-gebaseerde uitvoering, statusopvraging, snapshotondersteuning, gecontroleerd stoppen en foutafhandeling aan bod.

3.3.3. Onderzoeksmethoden

Deze fase combineerde analyse van het probleem met technische afleiding uit de projectdoelstelling. De vereisten werden niet alleen uit de literatuur afgeleid, maar ook uit de praktische workflow van het provisionen zelf. Daarbij werd nagegaan welke stappen absoluut nodig zijn om een VM automatisch klaar te zetten voor gebruik.

3.3.4. Verantwoording

Een duidelijke requirements-analyse was nodig om de scope van het project te bewaken. Zonder die afbakening zou de implementatie kunnen uitgroeien tot een algemeen VM-beheerhulpmiddel in plaats van een echte provisioner. De vereisten vormden daarom de basis voor alle latere sprints en voor de evaluatie van de uiteindelijke tool.

3.4. Fase 3: Proof of concept en sprintaanpak**3.4.1. Doelstelling**

In deze fase werd RustProv stapsgewijs gerealiseerd via een reeks sprints. De doelstelling was om de kernfunctionaliteit eerst werkend te krijgen en daarna uit te breiden met configuratie, extra beheermogelijkheden en foutafhandeling.

3.4.2. Deliverables

De belangrijkste deliverables waren werkende tussenversies van de tool, telkens met een uitbreidend functiebereik. Aan het einde van deze fase beschikte RustProv onder meer over functies voor opstarten, statuscontrole, provisioning via SSH, configuratie-inlezing, snapshots, priority handling en noodprocedures zoals kill en force stop.

3.4.3. Onderzoeksmethoden

Hier werd gekozen voor iteratieve ontwikkeling en experimenteel testen. Elke sprint bouwde voort op de vorige, waardoor nieuwe functionaliteiten gecontroleerd konden worden toegevoegd en gevalideerd. Deze aanpak liet toe om vroeg fouten of ontwerpkeuzes te detecteren, bijvoorbeeld rond UUID-conflicten, port forwarding en verschillen tussen Linux- en Windowsprovisioning.

3.4.4. Verantwoording

Een iteratieve aanpak is passend voor een proof of concept omdat de functionele basis eerst betrouwbaar moet werken voordat bijkomende functies zinvol zijn. Door per sprint werkende software op te leveren, kon het project tijdig worden bijgestuurd en bleef de implementatie beheersbaar.

3.5. Fase 4: Testopstelling en evaluatie**3.5.1. Doelstelling**

De vierde fase had als doel na te gaan of RustProv effectief bruikbaar is in de beoogde context en hoe de prestaties zich verhouden tot Vagrant. Hierbij werd gekeken naar zowel één-VM-scenario's als opstellingen met meerdere VM's.

3.5.2. Deliverables

Deze fase leverde een testopstelling op met representatieve Linux- en Windows-VM's, meetresultaten van provisioningtijden en een basis voor de vergelijking met de bestaande oplossing. Deze resultaten vormden de basis voor de analyse in het resultaten- en/of benchmarkhoofdstuk.

3.5.3. Onderzoeksmethoden

Er werd gebruikgemaakt van experimenteel onderzoek en tijdsmeting. De tests werden uitgevoerd onder gelijkwaardige omstandigheden, zodat de resultaten tussen RustProv en Vagrant zo goed mogelijk vergelijkbaar bleven. Daarnaast werd geobserveerd hoe de tool zich gedroeg in foutscenario's en bij verschillende configuraties.

3.5.4. Verantwoording

Testen zijn onmisbaar omdat de meerwaarde van een provisioner niet alleen afhangt van functionaliteit, maar ook van snelheid, stabiliteit en reproduceerbaarheid. Een proof of concept moet daarom niet alleen werken in theorie, maar ook aantoonbaar functioneel zijn in realistische scenario's.

3.6. Fase 5: Analyse van de resultaten

3.6.1. Doelstelling

In de laatste fase werden de testresultaten geïnterpreteerd en vertaald naar conclusies over de haalbaarheid van RustProv als alternatief voor de bestaande aanpak. Daarbij werd gekeken naar prestatieverschillen, praktische bruikbaarheid en beperkingen van de implementatie.

3.6.2. Deliverables

De deliverables van deze fase waren een geanalyseerde benchmarkvergelijking, een inhoudelijke bespreking van de sterktes en zwaktes van RustProv, en een onderbouwde conclusie over de mate waarin de tool de vooropgestelde doelstellingen haalt.

3.6.3. Onderzoeksmethoden

Voor deze fase werd vergelijkende analyse toegepast. De gemeten resultaten van RustProv werden naast die van Vagrant geplaatst en geïnterpreteerd in functie van de onderzoeksvraag. Daarnaast werd kwalitatief geëvalueerd welke keuzes in de implementatie geholpen hebben om de provisioningstijd en robuustheid te verbeteren.

3.6.4. Verantwoording

Een analysefase is nodig om de ruwe metingen betekenis te geven. Zonder interpretatie blijven meetresultaten louter cijfers; met analyse wordt duidelijk welke ontwerpkeuzes effectief impact hadden en waar de tool nog beperkingen vertoont.

3.7. Samenvatting

De gekozen methodologie combineerde literatuurstudie, requirements-analyse, iteratieve implementatie, experimenten en resultaatanalyse. Deze aanpak was geschikt omdat RustProv niet als afgebakend standaardproduct werd ontwikkeld, maar als een proof of concept dat in een beperkte tijdsspanne stapsgewijs moest worden opgebouwd en geëvalueerd. Door eerst de theoretische basis te leggen en daarna per sprint te werken, kon de ontwikkeling gericht en controleerbaar verlopen.

4

Iteratieve uitwerking

In dit hoofdstuk wordt de concrete evolutie van RustProv beschreven volgens de verschillende ontwikkelfasen. Elke fase bouwde voort op de vorige, waardoor de provisioner geleidelijk kon uitgroeien van een basisoplossing tot een tool met ondersteuning voor provisioning, configuratie, statusopvraging, snapshots en foutafhandeling. De nadruk lag telkens op het toevoegen van één afgebakende functionaliteit, waarna deze getest en verfijnd werd vooraleer de volgende stap werd uitgewerkt.

4.1. Fase 1: Rust aanleren

In de eerste fase werd de gekozen programmeertaal Rust aangeleerd. Daarbij lag de focus op basisconcepten zoals functies, datatypes, methodes en algemene syntax. Door kleine oefeningen en scripts uit te voeren werd de taal stap voor stap verkend, zodat de latere implementatie van de provisioner vlotter kon verlopen.

4.1.1. Week 1-2

Tijdens deze weken werd de Rust Quick Start Guide doorgenomen en werden verschillende scripts uitgevoerd om de belangrijkste concepten van de taal te oefenen (**Arbuckle2018**). Op die manier werd een basis gelegd voor de verdere ontwikkeling van het project.

4.2. Fase 2: Sprint 1 - opstarten

In de tweede fase werd gewerkt aan het opstarten van virtuele machines in VirtualBox. Het doel was om een VM automatisch te kunnen aanmaken, configureren en opstarten op basis van een vooraf gedefinieerde testbox. Deze fase vormde de technische basis van het volledige provisioningproces.

4.2.1. Week 3

Tijdens deze week werd het opstarten van een VM geïmplementeerd met behulp van verschillende VBoxManage-commando's. Er werd gestart met een Ubuntu- en een Windows 10-VM als basisboxen. Vervolgens werd het .vdi-bestand van de gekozen box gekopieerd en werd de UUID aangepast zodat meerdere VM's zonder conflict konden worden aangemaakt. Daarna werd de virtuele schijf gemount en werd de VM opgestart.

4.2.2. Week 4

Tijdens deze week werd de code herschreven zodat die beter aansloot bij een meer objectgeoriënteerde structuur. Hierdoor werd de code overzichtelijker en beter uitbreidbaar voor latere functies.

4.3. Fase 3: Sprint 2 - provisionen

In de derde fase werd het provisioningproces zelf uitgewerkt. Daarbij lag de focus op netwerktoegang via NAT-portforwarding en op het uitvoeren van scripts via SSH. Omdat Windows en Linux verschillend omgaan met scriptuitvoering, werd de logica opgesplitst per besturingssysteem.

4.3.1. Week 5

Tijdens deze week werd de VM verder aangepast zodat aparte functies beschikbaar waren voor Linux- en Windows-VM's. Hierdoor kon het gedrag van de provisioner worden afgestemd op het type gastbesturingssysteem.

4.3.2. Week 6

Tijdens deze week werd de verbinding met de VM via SSH verder uitgewerkt en werd de uitvoering van scripts per besturingssysteem toegepast. Daarmee werd het provisioningproces functioneel gemaakt voor zowel Linux- als Windowsomgevingen.

4.4. Fase 4: Sprint 3 - configuratie

In de vierde fase werd ondersteuning toegevoegd voor configuratiebestanden in TOML-formaat. Hierdoor kon het provisioninggedrag worden gestuurd vanuit een extern bestand in plaats van via hardgecodeerde waarden. Dat verhoogde de flexibiliteit en maakte de tool eenvoudiger aanpasbaar.

4.4.1. Week 7

Tijdens deze week werd het inlezen van het TOML-bestand opgestart. De implementatie was nog niet volledig afgerond, maar de basisstructuur werd wel al opgezet.

4.4.2. Week 8

Tijdens deze week werd het inlezen van het TOML-bestand verder uitgewerkt en geïntegreerd in het programma. Daarnaast werd een `init`-functie toegevoegd die de juiste folderstructuur aanmaakt, zodat de tool automatisch zijn noodzakelijke mappen kan voorbereiden.

4.5. Fase 5: Sprint 4 - functies

In de vijfde fase werden extra functies toegevoegd om de tool praktischer en robuuster te maken. Hierbij lag de focus op snapshots, het splitsen van functies voor één of meerdere VM's, het verbeteren van de CLI en het toevoegen van extra status- en netwerkfunctionaliteit.

4.5.1. Week 9

Tijdens deze week werd begonnen met het toepassen van snapshots. De code werd aangepast zodat het aanmaken van snapshots correct kon worden geïntegreerd in het provisioningproces. Daarnaast werden functies opgesplitst zodat ze afzonderlijk kunnen worden gebruikt voor één VM of voor meerdere VM's. Tot slot werd de CLI-input verder verbeterd met behulp van `clap`.

4.5.2. Week 10

Tijdens deze week werd een aparte functie toegevoegd om de status van de VM op te halen, zodat gecontroleerd kan worden of deze actief is. Daarnaast werd een bridge network adapter toegevoegd wanneer de gebruiker dat wenst. Tot slot werden de snapshotfuncties verder uitgewerkt en toegepast.

4.6. Fase 6: Sprint 5 - foutafhandeling

In de laatste fase lag de nadruk op foutafhandeling en rapportering. Het doel was om fouten duidelijk te signaleren en de gebruiker in geval van problemen niet achter te laten met een onbruikbare omgeving. Daarnaast werd nagedacht over fallback-acties zodat de tool zo stabiel mogelijk bleef werken.

4.6.1. Week 11

Tijdens deze week werd de functionaliteit voor priority queue toegevoegd, zodat VM's in een vooraf bepaalde volgorde kunnen worden opgestart en geprovisioned. Daarnaast werd de SSH-functionaliteit verder uitgewerkt, zodat gebruikers op een gebruiksvriendelijke manier verbinding kunnen maken met de virtuele machine zonder het IP-adres te weten van de client. Ook werd een kill-functie toegevoegd om VirtualBox geforceerd af te sluiten wanneer het programma niet meer correct reageert. Tot slot werden verschillende functies aangepast zodat het programma correct kan omgaan met VM's die al actief zijn.

4.6.2. Week 12

Tijdens deze week werd het provisioningproces voor Windows verder herwerkt. In de oorspronkelijke versie werd het script uitgevoerd vanuit een normale gebruiker met administratieve rechten, maar dit werd aangepast zodat via PSTools en psexec scripts als SYSTEM-gebruiker kunnen worden uitgevoerd. Deze wijziging was nodig omdat bepaalde scripts via SSH niet correct uitvoerbaar bleken onder een gewone beheerder. Daarnaast werd een force-stopfunctie toegevoegd voor Windows-VM's die niet correct wilden stoppen. Tot slot werd een script opgesteld om Windows-VM's automatisch voor te bereiden en psexec te installeren in de home folder van de gebruiker.

4.7. Planning

De totale duur van deze methodologie is gepland over een periode van 12 schoolweken tijdens semester 2 van het academiejaar 2025-2026.

- Week 1-2: Rust aanleren.
- Week 3-4: Sprint 1.
- Week 5-6: Sprint 2.
- Week 7-8: Sprint 3.
- Week 9-10: Sprint 4.
- Week 11-12: Sprint 5.

5

Requirements-analyse

Dit hoofdstuk beschrijft de minimale vereisten waaraan RustProv moet voldoen om als volwaardige provisioner te kunnen functioneren binnen de context van deze bachelorproef. De requirements-analyse vertrekt vanuit de algemene betekenis van provisioning, maar wordt vervolgens toegespitst op de concrete noden van de opleiding en de beoogde testomgevingen. Het doel van deze analyse is om vast te leggen welke functionaliteiten absoluut noodzakelijk zijn en welke uitbreidingen de tool praktischer en robuuster maken.

5.1. Minimale vereisten

Om als volwaardige provisioner te kunnen functioneren, moet **RustProv** minstens een aantal basisfuncties ondersteunen. Deze functies vormen samen de minimale keten die nodig is om een bruikbare virtuele omgeving automatisch op te zetten.

1. Configuratie kunnen inlezen

Een eerste vereiste is dat de tool configuratiegegevens kan inlezen uit een extern bestand. Hierdoor worden instellingen niet hardgecodeerd in de broncode en kan het gedrag van de tool aangepast worden zonder de implementatie te wijzigen. In RustProv gebeurt dit via TOML-configuratiebestanden, wat de leesbaarheid en uitbreidbaarheid van de configuratie verhoogt.

2. Een virtuele machine kunnen aanmaken of herkennen

Een provisioner moet in staat zijn om een virtuele machine aan te maken wanneer die nog niet bestaat, of een bestaande machine te herkennen wanneer die reeds aanwezig is. Deze vereiste is noodzakelijk om herbruikbare workflows te ondersteunen en om te vermijden dat telkens een volledig nieuwe omgeving moet worden opgebouwd.

3. Een VM kunnen starten en stoppen

Naast het aanmaken van een VM moet de tool deze ook gecontroleerd kunnen starten en stoppen. Zonder deze mogelijkheid kan het provisioningproces niet op een betrouwbare manier worden uitgevoerd. Voor RustProv is dit extra belangrijk omdat meerdere scenario's gebruikmaken van zowel één enkele VM als van een opstelling met meerdere onderling afhankelijke VM's.

4. Netwerktogang kunnen voorzien

Een provisioner moet de gastmachine kunnen bereiken via een netwerkverbinding, typisch via SSH of een gelijkaardig communicatiekanaal. Zonder deze stap is het niet mogelijk om automatisch configuratieopdrachten uit te voeren op de VM. In RustProv wordt hiervoor gewerkt met SSH-gebaseerde toegang, waarbij netwerkregels tijdelijk worden ingesteld zodat de machine bereikbaar wordt tijdens de provisioningfase.

5. Scripts of configuratiestappen kunnen uitvoeren

Het eigenlijke provisionen bestaat uit het uitvoeren van scripts of andere configuratiestappen op de gastmachine. Dit is de kernfunctie van de tool: zonder deze mogelijkheid blijft enkel VM-beheer over. De provisioner moet dus niet alleen toegang krijgen tot de machine, maar ook effectief een script kunnen starten en opvolgen tot de configuratie voltooid is.

6. Fouten kunnen signaleren en herstellen

Een laatste minimale vereiste is foutafhandeling. Wanneer een stap in het proces mislukt, moet de gebruiker geïnformeerd worden en moet de tool het proces gecontroleerd kunnen afbreken of herstellen. Dit is noodzakelijk om te vermijden dat een onbruikbare of half geconfigureerde omgeving achterblijft. In een onderwijscontext is dit extra belangrijk, omdat instabiele omgevingen leiden tot tijdverlies en verwarring.

5.2. Vereisten voor RustProv

Op basis van deze algemene minimale vereisten worden voor RustProv meer concrete functionele eisen geformuleerd. Deze vereisten zijn afgeleid uit de doelstelling van de bachelorproef en uit de nood aan ondersteuning voor zowel eenvoudige als meer complexe VM-scenario's binnen de opleiding.

Ondersteuning voor Windows en Linux RustProv moet zowel Windows- als Linux-VM's kunnen ondersteunen. Die keuze is logisch, omdat beide platformen binnen de opleiding gebruikt worden en omdat de bachelorproef expliciet gericht is op provisioning voor beide types gastbesturingssystemen.

Ondersteuning voor één of meerdere VM's De tool moet inzetbaar zijn voor zowel een eenvoudige opstelling met één VM als voor uitgebreidere scenario's

met meerdere VM's. Dat is nodig omdat de experimenten en benchmarks niet beperkt blijven tot één enkele machine, maar ook meerlagige opstellingen omvatten.

Externe configuratie via TOML Alle belangrijke instellingen moeten via een extern TOML-bestand kunnen worden beschreven. Op die manier wordt de opstelling declaratiever en eenvoudiger aanpasbaar. Deze vereiste ondersteunt ook de reproduceerbaarheid van experimenten, omdat dezelfde configuratie opnieuw gebruikt kan worden voor meerdere testruns.

SSH-gebaseerde uitvoering van scripts De uitvoering van provisioningsscripts moet via SSH of een vergelijkbare benadering gebeuren. RustProv gebruikt hiervoor een aparte SSH-library, omdat de ingebouwde oplossingen van Windows en Linux te veel van elkaar verschillen om op een uniforme manier te gebruiken. Deze keuze ondersteunt de cross-platform doelstelling van de tool.

VM-status kunnen opvragen De gebruiker moet de status van een VM kunnen opvragen. Deze functionaliteit is nodig om te bepalen of een machine reeds actief is, nog moet worden opgestart of correct werd afgesloten. Dit draagt bij tot een betrouwbaarder provisioningproces en voorkomt onnodige dubbele acties.

Snapshots kunnen maken en herstellen RustProv moet snapshots kunnen aanmaken en, waar nodig, gebruiken om sneller terug te keren naar een eerdere toestand. Deze vereiste is vooral relevant tijdens testen en herhaalde provisioningruns, omdat ze het mogelijk maakt om niet telkens een volledige VM opnieuw te moeten opbouwen.

VM's stoppen, geforceerd stoppen en verwijderen Naast gewone stopfunctionaliteit moet de tool ook ondersteuning bieden voor geforceerd stoppen en verwijderen van VM's. Dit is relevant omdat virtuele machines in de praktijk niet altijd correct reageren op standaardcommando's. Door deze mogelijkheden te voorzien, blijft de tool ook bruikbaar in foutscenario's.

Tijdelijke netwerkregels kunnen opschonen Omdat tijdens de provisioning tijdelijk netwerkregels kunnen worden toegevoegd, moet RustProv deze regels ook opnieuw kunnen verwijderen wanneer ze niet langer nodig zijn. Dit voorkomt conflicten met andere toepassingen en helpt om de hostomgeving netjes achter te laten na afloop van het proces.

5.3. Functionele en niet-functionele vereisten

De requirements van RustProv kunnen samengevat worden als een combinatie van functionele en niet-functionele vereisten.

Functionele vereisten

- Inlezen van configuratie uit een extern bestand.
- Aanmaken, herkennen, starten en stoppen van VM's.
- Uitvoeren van provisioning scripts op Windows- en Linux-VM's.
- Ondersteuning voor één of meerdere VM's binnen één scenario.
- Opvragen van VM-status, maken van snapshots en verwijderen van VM's.

Niet-functionele vereisten

- Herhaalbaarheid van experimenten en configuraties.
- Bruikbaarheid binnen een onderwijs- en labo-omgeving.
- Ondersteuning voor verschillende besturingssystemen.
- Betrouwbare foutafhandeling en gecontroleerd opschonen van tijdelijke configuraties.
- Voldoende flexibiliteit om nieuwe scenario's later toe te voegen.

5.4. Afbakening

Deze bachelorproef richt zich op een proof of concept en niet op een volledig productierijp provisioningplatform. Dat betekent dat de focus ligt op de kernfunctionaliteit die nodig is om geautomatiseerde Windows- en Linuxomgevingen te kunnen opzetten en vergelijken met een bestaande aanpak zoals Vagrant. Vereisten zoals uitgebreide logging, een grafische gebruikersinterface of brede ondersteuning voor andere hypervisors vallen buiten de primaire scope van dit onderzoek, tenzij ze rechtstreeks bijdragen aan de evaluatie van RustProv binnen de gekozen experimenten.

6

Toolselectie

Voor de realisatie van RustProv werd een aantal tools en bibliotheken geselecteerd op basis van drie centrale criteria: compatibiliteit met Windows en Linux, ondersteuning voor geautomatiseerde provisioning en voldoende flexibiliteit om zowel één VM als meerdere VM's te beheren. Daarnaast moest de gekozen stack aansluiten bij een onderwijscontext waarin herhaalbaarheid, stabiliteit en beheerbaarheid belangrijk zijn.

6.1. VirtualBox

Als virtualisatieplatform werd gekozen voor VirtualBox. Deze keuze sluit aan bij de doelomgeving van het project en maakt het mogelijk om virtuele machines programmatisch aan te maken, te configureren, te starten en te beheren. Bovendien ondersteunt VirtualBox de workflows die nodig zijn voor import, export en het gebruik van virtuele schijven, wat goed past bij de gekozen VDI-gebaseerde aanpak van RustProv.

Een bijkomend voordeel is dat VirtualBox reeds gebruikt wordt in de context waarop deze bachelorproef zich richt. Daardoor kon de provisioner ontwikkeld en getest worden in een omgeving die voldoende herkenbaar en relevant is voor de beoogde onderwijs- en labo-opstellingen.

6.2. Rust

Rust werd gekozen als programmeertaal omdat de provisioner op zowel Windows als Linux moet kunnen draaien zonder grote platformafhankelijke aanpassingen. Rust werd ontwikkeld voor efficiënte systeemsoftware en combineert prestatie met compile-time veiligheidsmechanismen, zoals ownership en borrowing, die bedoeld zijn om geheugen- en threadveiligheid te verbeteren. Dat maakt de taal

geschikt voor een infrastructuurtool die processen, configuraties en netwerkinteracties gecontroleerd moet beheren.

Voor RustProv is deze keuze relevant omdat betrouwbaarheid en voorspelbaar gedrag belangrijker zijn dan snelle prototyping alleen. Een provisioner moet immers niet alleen correct werken in het ideale scenario, maar ook robuust blijven bij fouttoestanden en verschillen tussen platformen.

6.3. **ssh2**

Voor de communicatie met de virtuele machines werd gekozen voor de `ssh2-crate`. Deze bibliotheek maakt het mogelijk om SSH-verbindingen op een uniforme manier te beheren vanuit Rust, wat belangrijk is in een tool die zowel Linux- als Windows-VM's moet kunnen provisionen.

De keuze voor een aparte SSH-bibliotheek was nodig omdat de ingebouwde SSH-oplossingen van Windows en Linux te veel van elkaar verschillen om op een consistente manier in één cross-platform tool te gebruiken. Door de SSH-logica centraal in Rust te beheren, werd de provisioningstroom eenvoudiger en beter controleerbaar.

6.4. **TOML en Serde**

Voor de configuratie van VM's en provisioninginstellingen werd gekozen voor TOML, in combinatie met de crates `toml` en `serde`. Deze combinatie maakt het mogelijk om configuratiegegevens op een leesbare en gestructureerde manier extern te bewaren en vervolgens automatisch om te zetten naar Rust-structuren.

Deze keuze is belangrijk omdat de configuratie van RustProv niet hardgecodeerd mocht blijven. Door een extern configuratiebestand te gebruiken, wordt de tool flexibeler, beter onderhoudbaar en geschikter voor herhaalbare experimenten.

6.5. **Clap**

Voor de command-line interface werd gekozen voor `clap`. Deze bibliotheek ondersteunt het gestructureerd verwerken van commando's, opties en argumenten, wat nuttig is voor een tool die meerdere acties moet aanbieden, zoals starten, provisionen, stoppen, verwijderen en het activeren van extra opties zoals snapshots of prioriteiten.

Dankzij deze keuze blijft de gebruikersinteractie overzichtelijk en uitbreidbaar. Dat is belangrijk voor RustProv, omdat de tool niet beperkt is tot één enkele actie, maar een volledige reeks beheerstappen binnen één command-line omgeving ondersteunt.

6.6. IndexMap

Voor het bewaren van de volgorde van VM-configuraties werd gekozen voor IndexMap. Deze datastructuur behoudt de invoervolgorde, wat relevant is in scenario's waarin de opstart- of provisioningsvolgorde van virtuele machines betekenis heeft. Die keuze is belangrijker dan bij een gewone HashMap, omdat een provisioner in meerlagige scenario's rekening moet kunnen houden met afhankelijkheden tussen VM's. Wanneer een bepaalde machine pas correct geconfigureerd kan worden nadat een andere reeds actief is, moet de verwerking in een voorspelbare volgorde kunnen verlopen.

6.7. Crossterm

Voor terminalinteractie en console-uitvoer werd `crossterm` gebruikt. Deze bibliotheek ondersteunt een duidelijkere en gebruiksvriendelijkere command-line ervaring, bijvoorbeeld voor statusmeldingen, foutmeldingen en voortgangsinformatie tijdens het uitvoeren van provisioningtaken.

Hoewel `crossterm` niet tot de kern van het provisioningsproces behoort, draagt deze keuze wel bij aan de bruikbaarheid van de tool. In een toepassing waarin veel stappen automatisch na elkaar verlopen, is heldere feedback naar de gebruiker een belangrijke meerwaarde.

6.8. PSTools

Voor Windows-specifieke provisioning werd gebruikgemaakt van PSTools, en meer bepaald van PsExec. Deze tool werd ingezet om scripts uit te voeren met verhoogde rechten wanneer dat via een gewone SSH-sessie niet volstond. Daardoor konden bepaalde administratieve configuratiestappen binnen Windows betrouwbaarder worden uitgevoerd.

Deze keuze was nodig omdat Windows-provisioning andere eisen stelt dan Linux-provisioning. Waar Linux-scripts doorgaans rechtstreeks via SSH en `sudo` kunnen worden uitgevoerd, vereiste de Windows-opstelling bijkomende ondersteuning om systeemniveau-acties op een consistente manier toe te passen.

7

Ontwerp

Dit hoofdstuk beschrijft het ontwerp van RustProv als provisioner voor virtuele Windows- en Linuxomgevingen. De nadruk ligt op de basisfuncties van de tool, de provisioningflow, de platformafhankelijke uitvoering van scripts en de uitbreidingen die nodig zijn om de tool bruikbaar te maken in realistische onderwijsomgevingen. Het ontwerp vertrekt vanuit het principe dat een provisioner niet enkel VM's moet kunnen starten, maar ook reproduceerbaar en gecontroleerd een omgeving moet kunnen voorbereiden.

7.1. Basisfunctionaliteit

RustProv werd ontworpen rond een beperkte set kernfuncties die samen de volledige provisioningworkflow ondersteunen. De belangrijkste kernfuncties zijn `start`, `provision`, `stop` en `remove`. Samen maken deze commando's het mogelijk om virtuele machines aan te maken, te configureren, af te sluiten en indien nodig opnieuw te verwijderen. Deze functies vormen de basis van de tool, omdat provisioning pas zinvol is wanneer een VM eerst correct kan worden opgestart en nadien op een gecontroleerde manier kan worden beheerd.

7.2. Gebruiksgemak en beheer

Naast de kernfuncties bevat RustProv verschillende ondersteunende functies die het gebruik van de tool eenvoudiger maken. De functies `state`, `ssh` en `init` verhogen vooral het gebruiksgemak en de controle over de virtuele omgeving. Met `state` kan de gebruiker opvragen of een VM actief is, `ssh` maakt rechtstreekse verbinding met de virtuele machine mogelijk en `init` zorgt voor het initialiseren van de nodige projectstructuur. Deze functies maken het beheer van de omgeving overzichtelijker en verminderen de kans op handmatige fouten.

7.3. Herstel- en noodfunctionaliteit

Tot slot bevat RustProv ook functies die bedoeld zijn voor herstel en foutafhandeling. De functie `restore` laat toe om een eerder opgeslagen snapshot opnieuw in te laden, terwijl `kill` als noodmaatregel wordt gebruikt wanneer VirtualBox niet meer correct reageert. Deze uitbreidingen zijn niet strikt noodzakelijk voor de basiswerking van de tool, maar verhogen wel de betrouwbaarheid en robuustheid van het systeem.

7.3.1. Provisioningworkflow

De provisioningworkflow volgt een vaste volgorde. Eerst wordt de configuratie ingelezen, daarna wordt de VM aangemaakt of herkend, vervolgens wordt netwerktoegang geconfigureerd en pas daarna wordt via SSH een script uitgevoerd op de gastmachine. Deze volgorde is bewust gekozen, omdat provisioning enkel betrouwbaar kan verlopen wanneer de VM eerst bereikbaar is en de juiste netwerkparameters beschikbaar zijn.

Wanneer meerdere VM's moeten worden verwerkt, worden eerst alle VM's opgestart en daarna pas de provisioningtaken uitgevoerd. Hierdoor wordt de totale wachttijd beperkt, omdat de provisioning niet per machine afzonderlijk hoeft te wachten tot de VM volledig beschikbaar is. Dit is vooral nuttig in scenario's met meerdere onderlinge afhankelijkheden.

7.4. SSH-verbinding

SSH vormt de centrale schakel tussen de host en de virtuele machine. RustProv gebruikt SSH om provisioningopdrachten op afstand uit te voeren. De gastmachine moet dus eerst bereikbaar worden gemaakt via het netwerk, doorgaans via port forwarding op poort 22, waarna de host verbinding kan maken met de VM.

Voordat de SSH-verbinding wordt opgezet, controleert RustProv eerst voor elke VM of poort 2222 reeds wordt doorgestuurd via port forwarding. Indien dat het geval is, wordt de bestaande regel verwijderd om conflicten te vermijden. Daarna wordt een nieuwe forwardingregel aangemaakt via poort 2222 op de host, zodat de VM op een gecontroleerde en voorspelbare manier bereikbaar is. Op die manier worden botsingen met andere virtuele machines of toepassingen die dezelfde poort gebruiken vermeden.

7.5. Linux- en Windowsprovisioning

RustProv splitst provisioning op in een Linux- en een Windows-pad. Die scheiding is nodig omdat beide besturingssystemen verschillende mechanismen gebruiken voor scriptuitvoering, rechtenbeheer en remote beheer. Een uniforme aanpak zou in de praktijk te veel beperkingen opleveren.

7.5.1. Linuxprovisioning

Bij Linuxprovisioning wordt het script via SSH uitgevoerd in combinatie met `sudo`. Het script bevindt zich in de `shared` folder en wordt vanuit de shell gestart. Deze aanpak sluit goed aan bij klassieke Linuxadministratie, waar remote scripting via SSH een standaardmethode is.

7.5.2. Windowsprovisioning

Windowsprovisioning vereist een andere aanpak. De scripts worden niet op dezelfde manier uitgevoerd als bij Linux, omdat de standaard SSH-aanpak niet altijd volstaat voor systeemwijzigingen. Daarom wordt een aparte uitvoeringsmethode gebruikt die meer overeenkomt met de vereisten van Windowsbeheer. De tool voert eerst de nodige netwerkconfiguratie uit, waarna het script in de gepaste context wordt gestart.

7.6. Uitbreidingen

Naast de basisfuncties bevat RustProv meerdere uitbreidingen die de tool robuuster en bruikbaar maken. Deze uitbreidingen werden toegevoegd omdat een provisioner in een labo-omgeving niet alleen correct moet werken, maar ook moet kunnen herstellen van fouten en zich moet aanpassen aan verschillende scenario's.

7.6.1. Snapshots

Snapshots werden toegevoegd om een antwoord te bieden op een praktisch probleem bij provisioning: wanneer een setup mislukt, moet men niet telkens opnieuw van nul beginnen. Door vóór de provisioning een snapshot te maken, kan de gebruiker terugkeren naar een stabiele toestand. Dat maakt testen sneller en verlaagt de impact van fouten tijdens de uitwerking.

7.6.2. Priority

Het prioriteitsmechanisme laat toe om VM's in een vooraf bepaalde volgorde te verwerken. Dat is nuttig wanneer bepaalde systemen pas correct kunnen functioneren nadat andere systemen al beschikbaar zijn. De volgorde wordt bijgehouden in de configuratie, zodat de provisioning reproduceerbaar blijft.

7.6.3. Stop, remove en restore

De functies `stop` en `remove` zijn voorzien om VM's gecontroleerd af te sluiten of volledig te verwijderen. De functie `restore` laat toe om een eerder opgeslagen snapshot terug te laden. Samen zorgen deze functies voor meer controle over de levenscyclus van een VM.

7.6.4. Kill

De `kill`-functie is een noodmaatregel voor situaties waarin VirtualBox niet meer correct reageert. Deze functie werd toegevoegd als een brute-force oplossing om vastgelopen toestanden te beëindigen wanneer normale stop- of beheercommando's geen resultaat meer opleveren. Op die manier kan RustProv alsnog terugkeren naar een werkbare toestand, zelfs wanneer de hypervisor niet meer op standaard-commando's reageert.

7.7. Ondersteuning voor Linux en Windows

RustProv werd ontworpen als een cross-platform toepassing die zowel op Linux als op Windows kan worden gecompileerd en uitgevoerd. Deze ondersteuning wordt gerealiseerd door functies conditioneel aan te passen op basis van het doelbesturingssysteem. Op die manier kan dezelfde codebasis op beide platformen gebruikt worden, terwijl platformgebonden onderdelen toch verschillend kunnen worden geïmplementeerd waar nodig. Hierdoor blijft de toepassing bruikbaar in uiteenlopende ontwikkel- en testomgevingen.

Een voorbeeld hiervan is de implementatie van het beëindigen van VirtualBox-processen. Op Windows wordt hiervoor gebruikgemaakt van `taskkill`, terwijl op Linux `pkill` wordt aangeroepen. Door deze logica af te stemmen op het besturingssysteem kan RustProv op beide platformen op een gepaste manier omgaan met processen die geforceerd moeten worden stopgezet.

Listing 7.1: Voorbeeld van platformafhankelijke procesafsluiting.

```
use std::process::Command;

pub fn main() {
    if cfg!(target_os = "windows") {
        let processes = ["VirtualBox.exe", "VBoxHeadless.exe", "VBoxSVC.exe"];

        for process in &processes {
            let _ = Command::new("taskkill")
                .args(&["/IM", process, "/F"])
                .status();
        }
    } else {
        let _ = Command::new("pkill")
            .args(&["-9", "-f", "VBox|VirtualBox"])
            .status();
    }
}
```


RustProv gebruikt een externe TOML-configuratie om per virtuele machine de nodige instellingen vast te leggen. Elke configuratie wordt gekoppeld aan een naam, zodat de verschillende VM's in een opstelling duidelijk van elkaar onderscheiden kunnen worden.

Listing 7.2: Voorbeeld van een TOML-configuratie voor een virtuele machine.

```
[ubuntu-wordpress]
image = "ubuntu"
cores = 4
memory = 4096
script = ["02-wordpress-promtail.sh"]
bridgeNetwork = "Realtek_Gaming_2.5GbE_Family_Controller"
priority = 1
```

Naast die naam bevat het configuratiebestand verschillende velden die elk een specifieke rol hebben binnen de opstelling. Het veld `image` bepaalt op welke basisimage de virtuele machine wordt gebaseerd. De velden `cores` en `memory` geven respectievelijk het aantal processorkernen en de hoeveelheid werkgeheugen weer. Met `script` wordt een lijst van uit te voeren scripts opgegeven. Daarnaast kan optioneel een `bridgeNetwork` worden ingevuld om een bridge-adapter te configureren, en kan `priority` worden gebruikt om de volgorde van opstarten of provisionen te bepalen. Op die manier blijft de configuratie leesbaar en kan de provisioninglogica de juiste VM op basis van haar naam en instellingen verwerken.

8

Implementatie

Dit hoofdstuk beschrijft hoe RustProv stapsgewijs werd opgebouwd tijdens de verschillende sprints van het project. Waar het hoofdstuk Methodologie vooral het plan van aanpak op hoofdlijnen toelicht, ligt de focus hier op de concrete technische uitwerking. Per sprint wordt aangegeven welke functionaliteiten werden gerealiseerd, welke ontwerpkeuzes daarbij belangrijk waren en wat de tool op dat moment reeds kon uitvoeren.

8.1. Sprint 1: opstarten van virtuele machines

De eerste implementatiesprint richtte zich op het automatisch kunnen aanmaken en opstarten van een virtuele machine in VirtualBox. Deze fase vormde de technische basis voor de rest van het project, omdat provisioning pas mogelijk wordt zodra een VM correct geregistreerd, geconfigureerd en gestart kan worden. In deze sprint werden de eerste functies `start` en `state` voorzien, al waren deze in het begin nog beperkt uitgewerkt.

Tijdens deze sprint werd gebruikgemaakt van `VBoxManage`-commando's om nieuwe VM's te registreren, de hardware-instellingen te configureren en een bestaand `.vdi`-bestand als virtuele schijf te koppelen. Daarbij werd een bestaande box gekopieerd naar een tijdelijke doelmap, waarna de UUID van de schijf aangepast werd. Deze stap was nodig om conflicten te vermijden wanneer meerdere VM's op basis van dezelfde box moesten worden aangemaakt. Daarnaast werd ook een shared folder gekoppeld, zodat latere provisioningsscripts vanuit de hostomgeving beschikbaar konden worden gemaakt in de gastmachine.

Aan het einde van deze sprint kon RustProv reeds een VM opstarten op basis van een voorbereide box en de toestand van die VM opvragen. De codebasis bestond aanvankelijk uit één groter bestand, maar werd tijdens deze fase al gedeeltelijk herschikt naar een meer modulaire structuur om latere uitbreidingen mogelijk te ma-

ken.

Listing 8.1: Voorbeeld van console-uitvoer na sprint 1.

8.2. Sprint 2: provisioning via SSH

De tweede sprint richtte zich op het effectieve provisioningproces. Nadat het opstarten van een VM functioneerde, werd de volgende stap het automatisch verbinden met de machine en het uitvoeren van scripts. Hierbij lag de nadruk op netwerktoegang via NAT-portforwarding en op de uitvoering van provisioningsscripts via SSH.

Omdat Linux- en Windowsomgevingen op een andere manier omgaan met scriptuitvoering, werd de provisioningslogica opgesplitst per type gastbesturingssysteem. Voor Linux werd een script uitgevoerd via `bash` en `sudo` vanuit de gemounte `shared` folder. Voor Windows werd een afwijkende aanpak gebruikt, waarbij via SSH een PowerShell-script gestart werd met behulp van `psexec`. Deze opsplitsing was nodig om binnen één tool toch beide platformen op een consistente manier te kunnen ondersteunen.

Een bijkomend aandachtspunt in deze sprint was het beheer van portforwarding-regels. Voor het opzetten van een SSH-verbinding werd een tijdelijke NAT-regel toegevoegd, zodat de host via `127.0.0.1:2222` de VM kon bereiken. Vooraf werd gecontroleerd of er reeds conflicterende regels bestonden, zodat poortbotsingen vermeden konden worden. Aan het einde van de provisioning werd deze tijdelijke netwerkregel opnieuw verwijderd.

Na deze sprint kon RustProv niet alleen een VM opstarten, maar ook automatisch verbinding maken via SSH en een provisioningscript uitvoeren op zowel Linux- als Windowsmachines. Daarmee werd de kern van de tool functioneel.

Listing 8.2: Voorbeeld van console-uitvoer bij Linux-provisioning na sprint 2.

Listing 8.3: Voorbeeld van console-uitvoer bij Windows-provisioning na sprint 2.

8.3. Sprint 3: externe configuratie

In de derde sprint werd ondersteuning toegevoegd voor configuratie via een extern TOML-bestand. In de eerdere versies van RustProv werden verschillende instellingen nog hardgecodeerd in de broncode. Dat beperkte de flexibiliteit van de tool en maakte het moeilijker om verschillende testscenario's op een reproduceerbare manier te beheren.

Door de introductie van TOML-configuratie konden VM-namen, besturingssysteemtypes, geheugeninstellingen, CPU-toewijzingen, boxkeuzes en bijkomende opties buiten de code worden beschreven. Deze configuratie werd ingelezen en omge-

zet naar interne datastructuren, waardoor éénzelfde uitvoerbare applicatie verschillende opstellingen kon behandelen zonder hercompilatie. In dezelfde sprint werd ook een `init`-functie toegevoegd om automatisch de vereiste mappenstructuur voor het project aan te maken.

Na deze sprint kon RustProv op basis van een declaratief configuratiebestand een VM-opstelling interpreteren en de nodige voorbereidingen treffen voor verdere uitvoering. Daarmee werd de tool niet alleen functioneel, maar ook beter bruikbaar voor herhaalde experimenten en verschillende onderwijsopstellingen.

Listing 8.4: Voorbeeld van console-uitvoer na het inlezen van TOML-configuratie.

8.4. Sprint 4: uitbreidingen en gebruiksgemak

De vierde sprint stond in het teken van uitbreidingen die de tool praktischer maakten in reële gebruiksscenario's. Een eerste belangrijke toevoeging was ondersteuning voor snapshots. Hiermee kon vóór een provisioningsrun een herstelpunt worden gemaakt, zodat bij fouten sneller teruggekeerd kon worden naar een stabiele toestand zonder de volledige VM opnieuw op te bouwen.

Daarnaast werd de code verder opgesplitst zodat bepaalde acties zowel voor één afzonderlijke VM als voor meerdere VM's konden worden toegepast. Ook de command-line interface werd verbeterd met `clap`, waardoor commando's en parameters duidelijker gestructureerd konden worden. In dezelfde sprint werd een afzonderlijke functie toegevoegd om de status van een VM op te vragen, zodat reeds actieve machines konden worden herkend en overgeslagen indien nodig. Verder werd optionele ondersteuning voorzien voor een bridge adapter als tweede netwerkinterface. Op het einde van deze sprint kon RustProv niet alleen configuraties inlezen en provisionen, maar ook scenario's met meerdere VM's beter beheren, snapshots aanmaken en meer bruikbare feedback geven aan de gebruiker via de CLI.

Listing 8.5: Voorbeeld van console-uitvoer voor snapshots en statuscontrole na sprint 4.

8.5. Sprint 5: foutafhandeling en robuustheid

De laatste sprint richtte zich op foutafhandeling, prioriteitsbeheer en robuuster gedrag in probleemsituaties. Daarbij werd een prioriteitsmechanisme toegevoegd zodat VM's in een vooraf bepaalde volgorde konden worden opgestart en geprovisioned. Dit was relevant voor scenario's waarin bepaalde machines pas correct konden functioneren nadat een andere reeds actief was. Om die reden werd ook gekozen voor `IndexMap`, zodat de invoervolgorde uit de configuratie behouden bleef. Daarnaast werden verschillende functies aangepast om correct om te gaan met reeds actieve VM's. De provisioner controleerde eerst de toestand van een machine en sloeg overbodige opstartstappen over wanneer de VM al draaide. Verder werd

een kill-functie toegevoegd als noodoplossing voor situaties waarin VirtualBox niet correct reageerde. Ook werden stop- en remove-functionaliteiten uitgebreid met geforceerde varianten, zodat vastgelopen VM's toch beheersbaar bleven.

Een belangrijk onderdeel van deze sprint was de verdere herwerking van Windows-provisioning. In de oorspronkelijke versie bleek scriptuitvoering onder een gewone gebruiker met administratieve rechten niet altijd voldoende. Daarom werd de aanpak aangepast zodat scripts via PSTools en psexec als SYSTEM-gebruiker konden worden uitgevoerd. Hierdoor werd Windows-provisioning betrouwbaarder voor systeemgerichte configuratiestappen.

Na deze sprint beschikte RustProv over een bruikbare proof of concept implementatie die VM's kon aanmaken, opstarten, provisionen, beheren en in foutscenario's gecontroleerd kon reageren. Daarmee was de basis gelegd voor de daaropvolgende experimenten en benchmarkvergelijkingen.

Listing 8.6: Voorbeeld van console-uitvoer na sprint 5 met meerdere VM's en prioriteit.

Listing 8.7: Voorbeeld van console-uitvoer bij foutafhandeling.

9

Experimentopzet

Om de werking van RustProv te evalueren, werd een testomgeving opgezet die aansluit bij een realistische onderwijscontext. Daarbij werd gekozen voor twee afzonderlijke scenario's: een Windows-omgeving en een Linux-omgeving. De Windowsomgeving is gebaseerd op een opstelling uit het vak Windows Server 2, terwijl de Linuxomgeving gebaseerd is op een opstelling uit het vak Software Development Project (SDP). Op die manier sluiten de experimenten aan bij bestaande en relevante praktijksituaties binnen de opleiding.

De testomgeving wordt gebruikt om RustProv te evalueren en te vergelijken met Vagrant als bestaande provisioningoplossing. Voor beide tools worden dezelfde virtuele machines, besturingssystemen, hardwareconfiguraties, netwerkinstellingen en provisioningscripts gebruikt. Hierdoor kunnen beide oplossingen onder zo gelijk mogelijke omstandigheden worden vergeleken.

De keuze voor deze opzet maakt het mogelijk om RustProv te testen in scenario's met één of meerdere virtuele machines, onderlinge afhankelijkheden en afzonderlijke provisioningscripts per machine. Op die manier kan niet alleen worden nagegaan of de tool technisch correct werkt, maar ook of RustProv bruikbaar is in een onderwijs- en labo-omgeving. De scenario's worden bovendien zo opgesteld dat zowel eenvoudige als complexere configuraties kunnen worden geëvalueerd.

De experimenten worden uitgevoerd voor één Linux-VM, meerdere Linux-VM's, één Windows-VM en meerdere Windows-VM's. Voor elk scenario wordt een overeenkomstige configuratie voorzien voor RustProv en Vagrant. De resultaten van deze experimenten worden in het volgende hoofdstuk weergegeven en besproken. Ook de manier waarop de verschillende tijdsmetingen worden uitgevoerd, wordt in het volgende hoofdstuk toegelicht.

9.1. Windows-omgeving

Voor het Windows-scenario werd een omgeving uitgewerkt die gebaseerd is op een opstelling uit het vak Windows Server 2. Deze omgeving bevat meerdere samenwerkende Windows-systemen binnen één netwerkstructuur. Ze wordt gebruikt om na te gaan of RustProv zowel een afzonderlijke Windows-VM als een complexere opstelling met meerdere Windows-systemen kan verwerken.

De Windowsomgeving wordt ook met Vagrant opgebouwd. Hierbij worden dezelfde VM-rollen, provisioningscripts en netwerkconfiguratie gebruikt als bij de RustProv-opstelling. Op die manier wordt een vergelijkingsbasis gecreëerd waarbij het verschil tussen beide oplossingen zo weinig mogelijk wordt beïnvloed door verschillen in de testomgeving.

9.1.1. Eén virtuele machine

windows

Provisioningscript

Het provisioningscript wordt gebruikt om de Windows-client voor te bereiden en de vereiste beheertools te installeren.

Listing 9.1: Provisioningscript voor de Windows-client.

% Het Windows-provisioningscript wordt hier ingevoegd.

9.1.2. Meerdere virtuele machines

windows

Provisioningscript voor Windows Server 1

Het eerste script configureert Windows Server 1 en bereidt de machine voor op de rol die deze binnen de testomgeving vervult.

Listing 9.2: Provisioningscript voor Windows Server 1.

% Het provisioningscript voor Windows Server 1 wordt hier ingevoegd.

Provisioningscript voor Windows Server 2

Het tweede script configureert Windows Server 2 en voorziet de instellingen die nodig zijn voor de samenwerking met de andere VM's.

Listing 9.3: Provisioningscript voor Windows Server 2.

% Het provisioningscript voor Windows Server 2 wordt hier ingevoegd.

Provisioningscript voor de Windows-client

Het derde script configureert de Windows-client binnen de meerledige Windows-omgeving.

Listing 9.4: Provisioningscript voor de Windows-client in de meerledige opstelling.

% Het provisioningscript voor de Windows-client wordt hier ingevoegd.

9.2. Linux-omgeving

Voor het Linux-scenario werd een opstelling voorzien met drie virtuele machines op basis van AlmaLinux. Deze omgeving is gebaseerd op een configuratie uit het vak Software Development Project (SDP) en werd gekozen omdat ze representatief is voor een typische Linux-opstelling binnen de opleiding.

De Linuxomgeving wordt zowel met RustProv als met Vagrant opgebouwd. Voor beide oplossingen worden dezelfde AlmaLinux-images, IP-adressen, hardware-instellingen en provisioningscripts gebruikt. Hierdoor kan de werking van RustProv onder dezelfde omstandigheden worden vergeleken met de bestaande Vagrant-aanpak.

9.2.1. Eén virtuele machine

Als eerste Linux-testgeval wordt één afzonderlijke AlmaLinux-VM gebruikt. Deze test heeft als doel te verifiëren of RustProv een enkele Linux-machine correct kan opstarten en provisionen. In tegenstelling tot de meerledige Linuxopstelling wordt in dit scenario een eenvoudig Bash-script gebruikt, zodat de basiswerking van de Linuxprovisioning afzonderlijk kan worden gecontroleerd.

Provisioningscript voor één Linux-VM

Het script voert enkele eenvoudige opdrachten uit op de gastmachine. Eerst wordt een tekstbericht naar de console geschreven. Vervolgens wordt een bestand aangemaakt waarin een tweede tekstbericht wordt opgeslagen. Ten slotte wordt de inhoud van dit bestand opnieuw weergegeven op de console. Op basis van deze uitvoer kan worden gecontroleerd of het script effectief op de juiste virtuele machine werd uitgevoerd. De configuratie van dit scenario wordt weergegeven in Listing 9.5.

Listing 9.5: Eenvoudig provisioningscript voor één Linux-VM.

```
#!/usr/bin/env bash
echo "Hello ,_World! "
echo "Hello ,_World_from_a_file!" > ~/hello.txt
cat ~/hello.txt
```

Voor het experiment met één Linux-VM werd dezelfde configuratie opgesteld voor RustProv en Vagrant. Beide gebruiken dezelfde AlmaLinux-image, VM-naam, hardware-

instellingen, netwerkadapter en het provisioningscript `00-hello.sh`. De configuraties worden weergegeven in Listing 9.6 en Listing 9.7.

Listing 9.6: RustProv-configuratie voor één Linux-VM.

```
[linuxbp]
image = "almalinux9"
cores = 4
memory = 4096
script = ["00-hello.sh"]
bridgeNetwork = "Realtek_Gaming_2.5GbE_Family_Controller"
```

Listing 9.7: Vagrant-configuratie voor één Linux-VM

```
Vagrant.configure("2") do |config|
  config.vm.define "linuxbp" do |linux|
    linux.vm.box = "almalinux/9"
    linux.vm.box_version = "9.7.20260518"
    linux.vm.hostname = "linuxbp"
    linux.vm.network "public_network", bridge: "Realtek_Gaming_2.5GbE_Family_Controller"
    linux.vm.provider "virtualbox" do |vb|
      vb.memory = 4096
      vb.cpus = 4
    end
    linux.vm.provision "shell", path: "scripts/00-hello.sh"
  end
end
```

9.2.2. Meerdere virtuele machines

Daarnaast wordt ook een Linux-opstelling met drie virtuele machines getest. Deze configuratie sluit aan bij een SDP-omgeving waarin meerdere AlmaLinux-systemen gelijktijdig worden ingezet. Het gebruik van meerdere Linux-VM's maakt het mogelijk om RustProv te evalueren in een scenario met meerdere nodes, afzonderlijke scripts en een gezamenlijke uitvoeringscontext.

De drie VM's hebben elk een eigen rol binnen de testomgeving. De eerste VM wordt gebruikt voor de database en Loki. De tweede VM bevat de WordPress-installatie en Promtail. De derde VM wordt gebruikt voor de monitoring met Grafana. De configuratie van deze VM's is onderling verbonden, omdat WordPress gebruikmaakt van de database op de eerste VM en Grafana de loggegevens uit Loki kan gebruiken.

Voor elke Linux-VM wordt een apart provisioningscript voorzien. Daardoor krijgt elke machine een eigen configuratie en rol binnen de bredere testomgeving. Deze aanpak maakt het mogelijk om te beoordelen of RustProv de juiste scriptstappen

op de juiste machine uitvoert en of meerdere Linux-systemen correct volgens de gewenste volgorde kunnen worden verwerkt.

Dezelfde drie VM-rollen, scripts en netwerkadressen worden ook in de Vagrantconfiguratie opgenomen. Hierdoor worden de RustProv- en Vagrantopstelling inhoudelijk gelijk gehouden.

Linux-script 1: database en Loki

Dit script installeert de databaseomgeving en Loki, zodat loggegevens centraal kunnen worden opgeslagen en later geanalyseerd. Het vormt de basislaag van de monitoringopstelling. De configuraties hiervan wordt weergegeven in Listing 9.8

Listing 9.8: Linux-script 1: installatie van database en Loki.

```
#!/usr/bin/env bash
set -euo pipefail

INTERFACE="eth1"

if nmcli connection show "${INTERFACE}" &>/dev/null; then
nmcli connection modify "${INTERFACE}" \
ipv4.addresses "192.168.0.10/24" \
ipv4.gateway "192.168.0.1" \
ipv4.dns "8.8.8.8,1.1.1.1" \
ipv4.method "manual"
else
nmcli connection add type ethernet con-name "${INTERFACE}" ifname "${INTERFACE}" \
ipv4.addresses "192.168.0.10/24" \
ipv4.gateway "192.168.0.1" \
ipv4.dns "8.8.8.8,1.1.1.1" \
ipv4.method "manual"
fi

nmcli connection up "${INTERFACE}"
sleep 5

dnf update -y
dnf install -y curl unzip mariadb-server polycoreutils-python-utils firewall

systemctl enable --now firewalld

firewall-cmd --permanent --add-service=mysql
firewall-cmd --permanent --add-port=3100/tcp
firewall-cmd --reload

if ! grep -q "^bind-address" /etc/my.cnf.d/mariadb-server.cnf 2>/dev/null; then
sed -i '/\[mysqld\]/a bind-address=0.0.0.0' /etc/my.cnf.d/mariadb-server.cnf
else
sed -i 's/^bind-address\s*=\s*/bind-address=0.0.0.0/' /etc/my.cnf.d/mariadb-s
fi
```

```
systemctl enable --now mariadb
```

```
DB_NAME="wordpress"
```

```
DB_USER="wp_user"
```

```
DB_PASS="password2026!"
```

```
WP_SERVER_IP="192.168.0.20"
```

```
mysql -u root <<EOF
```

```
CREATE DATABASE IF NOT EXISTS ${DB_NAME} DEFAULT CHARACTER SET utf8mb4 COLLATE
```

```
CREATE USER IF NOT EXISTS '${DB_USER}' '@'${WP_SERVER_IP} ' IDENTIFIED BY '${DB_P
```

```
GRANT ALL PRIVILEGES ON ${DB_NAME}.* TO '${DB_USER}' '@'${WP_SERVER_IP} ' ;
```

```
FLUSH PRIVILEGES;
```

```
EOF
```

```
LOKI_VERSION="2.9.2"
```

```
mkdir -p /etc/loki /var/loki
```

```
curl -sSL -O "https://github.com/grafana/loki/releases/download/v${LOKI_VERSION}
```

```
unzip -o loki-linux-amd64.zip
```

```
mv loki-linux-amd64 /usr/local/bin/loki
```

```
chmod +x /usr/local/bin/loki
```

```
rm -f loki-linux-amd64.zip
```

```
cat <<'EOF' > /etc/loki/loki-config.yml
```

```
auth_enabled: false
```

```
server:
```

```
http_listen_port: 3100
```

```
grpc_listen_port: 9096
```

```
common:
```

```
path_prefix: /var/loki
```

```
storage:
```

```
filesystem:
```

```
chunks_directory: /var/loki/chunks
```

```
rules_directory: /var/loki/rules
```

```
replication_factor: 1
```

```
ring:
```

```
kvstore:
```

```
store: inmemory
```

```
schema_config:
  configs:
    - from: 2020-05-15
  store: boltdb-shipper
  object_store: filesystem
  schema: v11
  index:
    prefix: index_
    period: 24h

  ruler:
    alertmanager_url: http://localhost:9093
EOF
```

```
chcon -t bin_t /usr/local/bin/loki 2>/dev/null || true
```

```
cat <<'EOF' > /etc/systemd/system/loki.service
```

```
[Unit]
```

```
Description=Loki service
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=root
```

```
ExecStart=/usr/local/bin/loki -config.file=/etc/loki/loki-config.yml
```

```
Restart=on-failure
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
systemctl daemon-reload
```

```
systemctl enable --now loki
```

Linux-script 2: WordPress en Promtail

Dit script installeert WordPress en Promtail. WordPress levert de webtoepassing, terwijl Promtail de logs van het systeem doorstuurt naar Loki voor centrale logverzameling. De configuraties hiervan wordt weergegeven in Listing 9.9

Listing 9.9: Linux-script 2: installatie van WordPress en Promtail.

```
#!/usr/bin/env bash
set -euo pipefail

INTERFACE="eth1"

nmcli con modify "$INTERFACE" ipv4.addresses 192.168.0.20/24 ipv4.gateway 192.
nmcli con up "$INTERFACE"
sleep 5

dnf install -y epel-release
dnf install -y nginx php-fpm php-mysqlnd php-cli php-curl php-gd php-mbstring

systemctl enable --now firewalld
firewall-cmd --permanent --add-service=http
firewall-cmd --reload

mkdir -p /var/www
curl -sSL https://wordpress.org/latest.tar.gz | tar -xz -C /var/www/

chown -R nginx:nginx /var/www/wordpress

DB_HOST="192.168.0.10"
DB_NAME="wordpress"
DB_USER="wp_user"
DB_PASS="password2026!"

cat <<EOF > /var/www/wordpress/wp-config.php
<?php
define( 'DB_NAME', '${DB_NAME}' );
define( 'DB_USER', '${DB_USER}' );
define( 'DB_PASSWORD', '${DB_PASS}' );
define( 'DB_HOST', '${DB_HOST}' );
define( 'DB_CHARSET', 'utf8mb4' );
define( 'DB_COLLATE', '' );
```

```
$(curl -sSL https://api.wordpress.org/secret-key/1.1/salt/)
```

```
\$table_prefix = 'wp_';
define( 'WP_DEBUG', false );

if ( ! defined( 'ABSPATH' ) ) {
    define( 'ABSPATH', __DIR__ . '/' );
}
require_once ABSPATH . 'wp-settings.php';
EOF
```

```
chown nginx:nginx /var/www/wordpress/wp-config.php
```

```
chcon -Rt httpd_sys_rw_content_t /var/www/wordpress
setsebool -P httpd_can_network_connect_db 1
setsebool -P httpd_can_network_connect 1
```

```
sed -i 's/user_=_apache/user_=_nginx/g' /etc/php-fpm.d/www.conf
sed -i 's/group_=_apache/group_=_nginx/g' /etc/php-fpm.d/www.conf
systemctl enable --now php-fpm
```

```
mv /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
```

```
cat <<'EOF' > /etc/nginx/nginx.conf
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;
include /usr/share/nginx/modules/*.conf;
events { worker_connections 1024; }
http {
    log_format main '$remote_addr_-$remote_user_[$time_local]_"$request"_'
        '$status_$body_bytes_sent_"$http_referer"_'
        '"$http_user_agent"_"$http_x_forwarded_for" ';
    access_log /var/log/nginx/access.log main;
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 4096;
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
```

```

        include /etc/nginx/conf.d/*.conf;
    }
EOF

server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;
    root /var/www/wordpress;
    index index.php index.html index.htm;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        try_files $uri =404;
        # AlmaLinux uses this socket path for PHP-FPM
        fastcgi_pass unix:/run/php-fpm/www.sock;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
    }
}
EOF

```

```

nginx -t
systemctl enable --now nginx

```

```

PROMTAIL_VERSION="2.9.2"
mkdir -p /etc/promtail /var/promtail

```

```

curl -sSL -O "https://github.com/grafana/loki/releases/download/v${PROMTAIL_VERSION}promtail-linux-amd64.zip"
unzip -o promtail-linux-amd64.zip
mv promtail-linux-amd64 /usr/local/bin/promtail
chmod +x /usr/local/bin/promtail
rm -f promtail-linux-amd64.zip

```

```

LOKI_IP="192.168.0.10"

```

```

cat <<EOF > /etc/promtail/promtail-config.yml

```



```

server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /var/promtail/positions.yaml

clients:
  - url: http://${LOKI_IP}:3100/loki/api/v1/push

scrape_configs:
  - job_name: nginx
    static_configs:
      - targets:
        - localhost
    labels:
      job: nginx_access
      host: wordpress-server
      __path__: /var/log/nginx/access.log
      - targets:
        - localhost
      labels:
        job: nginx_error
        host: wordpress-server
        __path__: /var/log/nginx/error.log
EOF

```

```
cat <<'EOF' > /etc/systemd/system/promtail.service
```

```
[Unit]
```

```
Description=Promtail service
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=root
```

```
ExecStart=/usr/local/bin/promtail -config.file=/etc/promtail/promtail-config.y
```

```
Restart=on-failure
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
systemctl daemon-reload  
systemctl enable --now promtail
```

Linux-script 3: monitoring met Grafana

Dit script installeert Grafana en maakt de monitoringomgeving volledig. Grafana gebruikt de data uit Loki om dashboards en logoverzichten weer te geven. De configuraties hiervan wordt weergegeven in Listing 9.10

Listing 9.10: Linux-script 3: installatie van Grafana en monitoring.

```
#!/usr/bin/env bash
set -euo pipefail

INTERFACE="eth1"

if nmcli con show "${INTERFACE}" >/dev/null 2>&1; then
nmcli con mod "${INTERFACE}" ipv4.addresses 192.168.0.30/24 ipv4.gateway 192.1
else
nmcli con add type ethernet con-name "${INTERFACE}" ifname "${INTERFACE}" ipv4
fi

nmcli con up "${INTERFACE}"
sleep 5

dnf install -y wget curl unzip firewallld

systemctl enable --now firewallld
firewall-cmd --permanent --add-port=3000/tcp
firewall-cmd --reload

# Create the RPM repository file for Grafana
cat <<EOF > /etc/yum.repos.d/grafana.repo
[grafana]
name=grafana
baseurl=https://rpm.grafana.com
repo_gpgcheck=1
enabled=1
gpgcheck=1
gpgkey=https://rpm.grafana.com/gpg.key
sslverify=1
sslcacert=/etc/pki/tls/certs/ca-bundle.crt
EOF

dnf install -y grafana

LOKI_IP="192.168.0.10"
```

```
mkdir -p /etc/grafana/provisioning/datasources
```

```
cat <<EOF > /etc/grafana/provisioning/datasources/loki.yaml
apiVersion: 1
```

```
datasources:
- name: Loki
type: loki
access: proxy
url: http://${LOKI_IP}:3100
isDefault: true
editable: true
EOF
```

```
PROMTAIL_VERSION="2.9.2"
mkdir -p /etc/promtail /var/promtail
```

```
curl -sSL -O "https://github.com/grafana/loki/releases/download/v${PROMTAIL_VERSION}
unzip -o promtail-linux-amd64.zip
mv promtail-linux-amd64 /usr/local/bin/promtail
chmod +x /usr/local/bin/promtail
rm -f promtail-linux-amd64.zip
```

```
cat <<EOF > /etc/promtail/promtail-config.yml
server:
http_listen_port: 9080
grpc_listen_port: 0
```

```
positions:
filename: /var/promtail/positions.yaml
```

```
clients:
- url: http://${LOKI_IP}:3100/loki/api/v1/push
```

```
scrape_configs:
- job_name: varlogs
static_configs:
- targets:
- localhost
labels:
job: varlogs
```

```
host: monitoring-server
__path__: /var/log/*.log
EOF
```

```
cat <<'EOF' > /etc/systemd/system/promtail.service
```

```
[Unit]
```

```
Description=Promtail service
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=root
```

```
ExecStart=/usr/local/bin/promtail -config.file=/etc/promtail/promtail-config.y
```

```
Restart=on-failure
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
systemctl daemon-reload
```

```
systemctl enable --now grafana-server
```

```
systemctl enable --now promtail
```

Voor de Linuxexperimenten werd dezelfde VM-opstelling beschreven in zowel een TOML-configuratiebestand voor RustProv als een Vagrantfile voor Vagrant. Op die manier wordt ervoor gezorgd dat beide provisioners dezelfde virtuele machines, hardware-instellingen, netwerkadapter en provisioningscripts gebruiken. De configuraties worden weergegeven in Listing 9.11 en Listing 9.12.

Listing 9.11: RustProv-configuratie voor meerdere Linux-VM's.

```
[dbloki]
image = "almalinux9"
cores = 4
memory = 4096
script = ["01-db-loki.sh"]
bridgeNetwork = "Realtek_Gaming_2.5GbE_Family_Controller"

[wppromtail]
image = "almalinux9"
cores = 4
memory = 4096
script = ["02-wordpress-promtail.sh"]
bridgeNetwork = "Realtek_Gaming_2.5GbE_Family_Controller"

[monitorgrafana]
image = "almalinux9"
cores = 4
memory = 4096
script = ["03-monitoring-grafana.sh"]
bridgeNetwork = "Realtek_Gaming_2.5GbE_Family_Controller"
```

Listing 9.12: Vagrant-configuratie voor meerdere Linux-VM's.

```
Vagrant.configure("2") do |config|
  config.vm.box = "almalinux/9"
  config.vm.box_version = "9.7.20260518"
  config.vm.define "dbloki" do |dbloki|
    dbloki.vm.hostname = "dbloki"
    dbloki.vm.network "public_network", bridge: "Realtek_Gaming_2.5GbE_Family_Con
    dbloki.vm.provider "virtualbox" do |vb|
      vb.memory = 4096
      vb.cpus = 4
    end
    dbloki.vm.provision "shell", path: "scripts/01-db-loki.sh"
  end
  config.vm.define "wppromtail" do |wp|
    wp.vm.hostname = "wppromtail"
    wp.vm.network "public_network", bridge: "Realtek_Gaming_2.5GbE_Family_Control
    wp.vm.provider "virtualbox" do |vb|
      vb.memory = 4096
      vb.cpus = 4
    end
    wp.vm.provision "shell", path: "scripts/02-wordpress-promtail.sh"
  end
  config.vm.define "monitorgrafana" do |grafana|
    grafana.vm.hostname = "monitorgrafana"
    grafana.vm.network "public_network", bridge: "Realtek_Gaming_2.5GbE_Family_Co
    grafana.vm.provider "virtualbox" do |vb|
      vb.memory = 4096
      vb.cpus = 4
    end
    grafana.vm.provision "shell", path: "scripts/03-monitoring-grafana.sh"
  end
end
```

TOE TE VOEGEN 1 LINUX MULTI LINUX 1 WINDOWS MULTI WINDOWS

Tijd testen na elkaar

Tijd testen apart -> aanmaken vm -> daarna provisionen niet aanmaken en provi-
soinen

TOML voor rustpov Vagrantfile voor vagrant voor de config

10

Resultaten

Dit hoofdstuk bespreekt de resultaten van de experimenten uit Hoofdstuk 9. Eerst wordt toegelicht hoe de uitvoeringstijden werden gemeten. Daarbij wordt een onderscheid gemaakt tussen scenario's met één virtuele machine en scenario's met meerdere virtuele machines. Vervolgens worden de metingen voor Linux en Windows weergegeven. Ten slotte worden de resultaten van RustProv en Vagrant met elkaar vergeleken.

10.1. Meetmethode

Om de uitvoeringstijden op een consistente manier te registreren, werden twee PowerShell-scripts gebruikt. Het eerste script is bedoeld voor scenario's met één virtuele machine. Het tweede script wordt gebruikt voor scenario's met meerdere virtuele machines. Beide scripts meten de duur van een opgegeven opdracht en schrijven het resultaat weg naar een CSV-bestand.

Bij één virtuele machine worden de starttijd en de provisioningtijd afzonderlijk gemeten. De starttijd is de tijd die nodig is om de VM op te starten. De provisioningtijd is de tijd die nodig is om het provisioningsscript uit te voeren.

Bij meerdere virtuele machines wordt alleen de totale uitvoeringstijd van de volledige opstelling gemeten. Deze tijd omvat zowel het opstarten als het provisionen van alle VM's. De VM's worden daarbij na elkaar verwerkt.

Elke meting wordt drie keer uitgevoerd onder dezelfde omstandigheden. Hierdoor wordt vermeden dat één uitzonderlijk snelle of trage uitvoering het resultaat bepaalt. Op basis van de drie afzonderlijke metingen wordt per scenario het gemiddelde berekend. Dit gemiddelde wordt gebruikt om RustProv en Vagrant met elkaar te vergelijken.

Voor één virtuele machine wordt de totale uitvoeringstijd berekend als de som van de gemiddelde starttijd en de gemiddelde provisioningtijd:

$$\bar{T}_{\text{totaal}} = \bar{T}_{\text{start}} + \bar{T}_{\text{provisioning}}$$

Voor meerdere virtuele machines wordt de totale uitvoeringstijd rechtstreeks gemeten vanaf het starten van de volledige opstelling tot het moment waarop alle VM's succesvol zijn geprovisioned.

Dezelfde meetmethode wordt toegepast op RustProv en Vagrant. De gebruikte virtuele machines, configuraties, provisioning-scripts en hardware-instellingen blijven daarbij gelijk.

10.1.1. PowerShell-meetscripts

Voor de meting van één virtuele machine wordt een afzonderlijk PowerShell-script gebruikt. Dit script meet de starttijd en de provisioningtijd afzonderlijk. Het volledige script wordt weergegeven in Listing 10.1.

Listing 10.1: PowerShell-script voor het meten van één virtuele machine.

% Het PowerShell-meetscript voor één VM wordt hier ingevoegd.

Voor scenario's met meerdere virtuele machines wordt een tweede PowerShell-script gebruikt. Dit script meet enkel de totale uitvoeringstijd van de volledige opstelling. Het volledige script wordt weergegeven in Listing 10.2.

Listing 10.2: PowerShell-script voor het meten van meerdere virtuele machines.

% Het PowerShell-meetscript voor meerdere VM's wordt hier ingevoegd.

Beide scripts registreren per meting minstens de gebruikte tool, het scenario en de uitvoeringstijd. De tijd wordt uitgedrukt in seconden. De resultaten worden opgeslagen in een CSV-bestand, zodat de afzonderlijke metingen kunnen worden verwerkt en het gemiddelde per scenario kan worden berekend.

10.2. Meting van één Linux-VM

Voor het eerste scenario werd één AlmaLinux-VM gebruikt. De starttijd en de provisioningtijd werden afzonderlijk gemeten voor zowel RustProv als Vagrant. De VM werd voor elke nieuwe meting in dezelfde beginsituatie geplaatst.

Bij RustProv werd de starttijd gemeten met het commando `start`. Nadat de VM was opgestart, werd de provisioningtijd gemeten met het commando `provision`. Bij Vagrant werd dezelfde procedure gevolgd met respectievelijk `vagrant up` en `vagrant provision`.

Tabel 10.1 toont de afzonderlijke metingen en de gemiddelde uitvoeringstijden voor RustProv en Vagrant bij het scenario met één Linux-VM. De totale uitvoeringstijd wordt berekend door de gemiddelde starttijd en de gemiddelde provisioningtijd bij elkaar op te tellen.

Tabel 10.1: Meting van één Linux-VM met RustProv en Vagrant.

Tool	Onderdeel	Run 1 (s)	Run 2 (s)	Run 3 (s)	Gemiddelde (s)
RustProv	Start	–	–	–	–
RustProv	Provisioning	–	–	–	–
Vagrant	Start	–	–	–	–
Vagrant	Provisioning	–	–	–	–

10.3. Meting van meerdere Linux-VM's

Voor het tweede scenario werden drie AlmaLinux-VM's gebruikt. De VM's werden één voor één opgestart en vervolgens één voor één geprovisioned. In tegenstelling tot het scenario met één Linux-VM wordt hier alleen de totale uitvoeringstijd van de volledige Linuxopstelling gemeten.

De meting start wanneer het commando voor de volledige opstelling wordt uitgevoerd en eindigt wanneer alle drie de VM's zijn opgestart en de bijhorende provisioningsscripts succesvol na elkaar zijn uitgevoerd. De VM's werden tijdens elke run in dezelfde volgorde verwerkt, zodat de uitvoeringsvolgorde tussen de verschillende metingen gelijk bleef.

Tabel 10.2: Totale uitvoeringstijd van meerdere Linux-VM's met RustProv en Vagrant.

Tool	Run 1 (s)	Run 2 (s)	Run 3 (s)	Gemiddelde (s)
RustProv	–	–	–	–
Vagrant	–	–	–	–

Tabel 10.2 toont de drie afzonderlijke metingen en het gemiddelde voor de volledige Linuxopstelling. De gemeten tijd omvat zowel het na elkaar opstarten van de drie VM's als het na elkaar uitvoeren van de bijhorende provisioningsscripts.

10.4. Meting van één Windows-VM

Voor het derde scenario werd één Windows-VM gebruikt. De starttijd en de provisioningtijd werden afzonderlijk gemeten voor RustProv en Vagrant. Voor elke run werd dezelfde Windows-VM gebruikt en werd dezelfde beginsituatie voorzien.

De starttijd omvat het opstarten van de Windows-VM. De provisioningtijd omvat het uitvoeren van het Windows-provisioningscript en het afronden van de bijhorende configuratiestappen.

Tabel 10.3 toont de afzonderlijke metingen en de gemiddelde uitvoeringstijden voor RustProv en Vagrant bij het scenario met één Windows-VM. De gemiddelde waarden worden gebruikt om de uitvoeringstijd van beide provisioners met elkaar

Tabel 10.3: Meting van één Windows-VM met RustProv en Vagrant.

Tool	Onderdeel	Run 1 (s)	Run 2 (s)	Run 3 (s)	Gemiddelde (s)
RustProv	Start	–	–	–	–
RustProv	Provisioning	–	–	–	–
Vagrant	Start	–	–	–	–
Vagrant	Provisioning	–	–	–	–

te vergelijken.

10.5. Meting van meerdere Windows-VM's

Voor het vierde scenario werden drie Windows-VM's gebruikt: twee servers en één client. De VM's werden tijdens elke run in dezelfde volgorde opgestart en geprovisioned. Net zoals bij het meerledige Linuxscenario wordt hier alleen de totale uitvoeringstijd van de volledige Windowsopstelling gemeten.

De meting start wanneer het commando voor de volledige Windowsopstelling wordt uitgevoerd en eindigt wanneer alle drie de VM's zijn opgestart en de drie Windowsprovisioningsscripts na elkaar succesvol zijn uitgevoerd.

Tabel 10.4: Totale uitvoeringstijd van meerdere Windows-VM's met RustProv en Vagrant.

Tool	Run 1 (s)	Run 2 (s)	Run 3 (s)	Gemiddelde (s)
RustProv	–	–	–	–
Vagrant	–	–	–	–

Tabel 10.4 toont de drie afzonderlijke metingen en het gemiddelde voor de volledige Windowsopstelling. De gemeten tijd omvat zowel het na elkaar opstarten van de drie Windows-VM's als het na elkaar uitvoeren van de drie Windowsprovisioningsscripts.

10.6. Vergelijking tussen RustProv en Vagrant

11

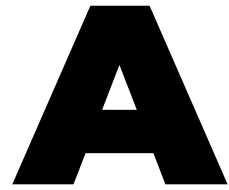
Conclusie

Curabitur nunc magna, posuere eget, venenatis eu, vehicula ac, velit. Aenean ornare, massa a accumsan pulvinar, quam lorem laoreet purus, eu sodales magna risus molestie lorem. Nunc erat velit, hendrerit quis, malesuada ut, aliquam vitae, wisi. Sed posuere. Suspendisse ipsum arcu, scelerisque nec, aliquam eu, molestie tincidunt, justo. Phasellus iaculis. Sed posuere lorem non ipsum. Pellentesque dapibus. Suspendisse quam libero, laoreet a, tincidunt eget, consequat at, est. Nullam ut lectus non enim consequat facilisis. Mauris leo. Quisque pede ligula, auctor vel, pellentesque vel, posuere id, turpis. Cras ipsum sem, cursus et, facilisis ut, tempus euismod, quam. Suspendisse tristique dolor eu orci. Mauris mattis. Aenean semper. Vivamus tortor magna, facilisis id, varius mattis, hendrerit in, justo. Integer purus.

Vivamus adipiscing. Curabitur imperdiet tempus turpis. Vivamus sapien dolor, congue venenatis, euismod eget, porta rhoncus, magna. Proin condimentum pretium enim. Fusce fringilla, libero et venenatis facilisis, eros enim cursus arcu, vitae facilisis odio augue vitae orci. Aliquam varius nibh ut odio. Sed condimentum condimentum nunc. Pellentesque eget massa. Pellentesque quis mauris. Donec ut ligula ac pede pulvinar lobortis. Pellentesque euismod. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent elit. Ut laoreet ornare est. Phasellus gravida vulputate nulla. Donec sit amet arcu ut sem tempor malesuada. Praesent hendrerit augue in urna. Proin enim ante, ornare vel, consequat ut, blandit in, justo. Donec felis elit, dignissim sed, sagittis ut, ullamcorper a, nulla. Aenean pharetra vulputate odio.

Quisque enim. Proin velit neque, tristique eu, eleifend eget, vestibulum nec, lacus. Vivamus odio. Duis odio urna, vehicula in, elementum aliquam, aliquet laoreet, tellus. Sed velit. Sed vel mi ac elit aliquet interdum. Etiam sapien neque, convallis et, aliquet vel, auctor non, arcu. Aliquam suscipit aliquam lectus. Proin tincidunt magna sed wisi. Integer blandit lacus ut lorem. Sed luctus justo sed enim.

Morbi malesuada hendrerit dui. Nunc mauris leo, dapibus sit amet, vestibulum et, commodo id, est. Pellentesque purus. Pellentesque tristique, nunc ac pulvinar adipiscing, justo eros consequat lectus, sit amet posuere lectus neque vel augue. Cras consectetur libero ac eros. Ut eget massa. Fusce sit amet enim eleifend sem dictum auctor. In eget risus luctus wisi convallis pulvinar. Vivamus sapien risus, tempor in, viverra in, aliquet pellentesque, eros. Aliquam euismod libero a sem. Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Binnen de richting IT infrastructure engineer wordt in HOGENT gebruikgemaakt van verschillende provisioningtools om virtuele Windows- en Linuxomgevingen op te zetten. Een veelgebruikte tool hiervoor is Vagrant, studenten gebruiken dit om een ontwikkel- en labo-omgeving te kunnen automatiseren op basis van een declaratieve configuratiefile.

In de praktijk komen er verschillende problemen tevoorschijn: de virtuele machines starten soms traag op, het provisioningproces hangt vast of kan niet gestart worden door time-outs, virtuele machines kunnen onverwachte kernel panics vertonen en als het provisioningproces onderbroken wordt tijdens uitvoer blijft het soms verder draaien op de achtergrond waardoor de gebruiker dit proces manueel moet stoppen om verder te kunnen werken. Dit zorgt voor vele frustraties en tijdverlies zowel binnen een onderwijscontext of bedrijfscontext.

De centrale onderzoeksvraag van deze bachelorproef is: "Hoe ontwerp en realiseer je een performante provisioner voor een virtuele Windows- en Linux omgeving, die beter inspeelt op de behoeften van onderwijs en bedrijven dan de huidige Vagrant-gebaseerde aanpak?" Om deze vraag te beantwoorden doen we aan de hand van deze deelvragen:

- Welke functionaliteiten zijn nodig voor een provisioner?
- Op welke besturingssystemen zal de provisioner gebruikt worden?
- Voor welke gast-besturingssystemen provisioned de tool?

Dit onderzoek streeft ervoor om een provisioner te maken die gebruikt kan worden door IT-professionals en studenten binnen een onderwijs- en labo-omgevingen. Om deze vragen te beantwoorden wordt een sprint-aanpak gebruikt, dit loopt over 12 weken: eerst studeren van de programmeertaal Rust, daarna stapsgewijs het kunnen opstarten van de Virtuele Machine, Provisioning via SSH, Configuratie via TOML, snapshots en foutafhandeling implementeren. Elke sprint bouwt verder op de vorige en zal getest worden aan de hand van testscenario's die gebaseerd zijn op HOGENT labo oefeningen

A.2. Literatuurstudie

Virtuele machines vormen de basis van veel Informaticatoepassingen. De voorname reden hiervoor is omdat ze het mogelijk maken om meerdere virtuele omgevingen te laten draaien op dezelfde fysieke hardware en zo de flexibiliteit, schaalbaarheid en resource-efficiëntie verhogen (Ahmed2020). In verschillende surveys wordt *provisioning* beschreven als het geautomatiseerd aanmaken, configureren en beheren van een virtuele machine (Kirana2018; Abdulhamid2014). Zoals deze studies aangeven is het opzetten van een testomgeving met efficiënte provisioning-strategieën cruciaal bij het uitwerken van een project.

A.2.1. Infrastructure as Code

Het automatiseringsproces wordt in systeembeheer genoemd *Infrastructure as Code (IaC)*. Verschillende studies en rapporten binnen het IaC een maken onderscheid tussen verschillende soorten tools, zoals *emphprovisioning tools*, *configuration management* en *orchestration* (Hill2015; Yevgeniy2025). Provisioningtools richten zich op het creëren van de infrastructuur, configuratiemanagement tools richten zich op de installatie en configuratie van software op de server (Ijaz2024). Het landschap van deze verschillende tools is divers. Voor de lokale ontwikkelingsomgevingen wordt gebruikt gemaakt van tools zoals Vagrant. Verschillende tools bestaan ook die dat niet gemaakt zijn als eerste instantie voor lokale omgevingen zoals Ansible en Terraform, deze worden vaak geassocieerd met cloud omgevingen of grote configuraties (Ismailzai2017).

A.2.2. Educatieve context

Alhoewel dat er veel verschillende commerciële oplossingen bestaan voor enterprise-omgevingen (Portworx2025), is er binnen een onderwijscontext andere verschillende eisen voor virtualisatie. Onderzoek naar het gebruik van virtuele machines (VM's) in het onderwijs toont dat een balans tussen performantie, gebruiksvriendelijkheid voor studenten en de beperkingen van consumenten hardware zoals laptops een complexe uitdaging is (Robison2012). Ook speelt tijd een grote rol. De *provisioning time*, de tijd die tussen het aanvra-

Figuur A.1: Gantt diagram met de verschillende fasen en milestones van het onderzoek.

gen van de vm en beschikbaarheid ervan is kritieke performance-indicator (KPI) die dat productiviteit beïnvloed (**KpiDepot2024**). Lange provisioning time kan tijdens het maken van een labo of examen voor minder werktijd zorgen. Gelukkig zijn er wel verschillende algoritmen die dat dit kunnen versnellen (**jcai2020p208; Lo2014**), blijft de vraag bestaan hoe je een lichtgewicht en snelle provisioner kan gerealiseerd worden die dat specifiek gemaakt is voor Windows- en Linux-omgevingen voor een educatieve omgeving als alternatief voor de huidige Vagrant provisioner.

A.2.3. Rust als programmeertaal

Rust werd gekozen voor de provisioner vanwege goede cross-platform compatibiliteit, waardoor de tool zowel op Linux als Windows werkt zonder veel aanpassingen te doen. De taal combineert performantie en geheugenveiligheid door het Ownership-model, dit zorgt ervoor dat veel voorkomende fouten zoals geheugenlekken voorkomen worden (**Micah2023; Klabnik2018**). Dit is cruciaal voor infrastructuurtools, bij het afhandelen van foutscenario's zoals een onderbroken provisioning of een mislukte SSH-verbinding zorgt dit ervoor dat alles onder controle blijft (**Micah2023**).

A.3. Methodologie

A.3.1. Fase 1: Rust aanleren

In de eerste fase wordt de focus gelegd op het aanleren van de gekozen programmeertaal Rust. Hierbij zullen de basisconcepten van de taal geleerd worden, zoals functies, methodes, datatypes en algemene syntax. Aan de hand van verschillende kleine projecten worden deze concepten dan toegepast en aangeleerd.

A.3.2. Fase 2: Sprint 1 - opstarten

In de eerste sprint ligt de focus op het kunnen opstarten van de virtuele machine (VM) op basis van een gemaakte testbox. De provisioner zal een VM kunnen aanmaken, configureren en starten met het virtualisatieplatform VirtualBox.

A.3.3. Fase 3: Sprint 2 - provisionen

In de tweede stap zal het provisioningproces worden uitgewerkt. De tool zal het IP-adres opvragen van de VM. Daarna wordt via SSH verbinding gemaakt en vervolgens de provisioningstappen uitgevoerd op het systeem.

A.3.4. Fase 4: Sprint 3 - configuratie

In de derde sprint wordt er ondersteuning toegevoegd voor een configuratiebestand in het TOML-formaat. De provisioner zal het configuratiebestand inlezen en verwerken, de gevraagde instellingen zullen het provisioninggedrag besturen.

A.3.5. Fase 5: Sprint 4 - functies

In de vierde sprint zullen nieuwe functies worden toegevoegd, zoals in het aanmaken en terugzetten van snapshots. Dit zorgt er voor dat de gebruiker snel terug zal kunnen gaan naar een stabiele toestand, zoals na een mislukte provisioningrun. Ook zal tijdens deze sprint meerdere boxes aangemaakt worden.

A.3.6. Fase 6: Sprint 5 - foutafhandeling

In de laatste sprint ligt de focus op het bouwen van foutafhandeling en rapportering. Fouten worden gelogd met duidelijke foutboodschappen en genoteerd in een log file. Als er een fout is moet er ook op een veilige manier een fallback-actie uitgevoerd worden, dit zorgt ervoor dat de gebruiker niet met een onbruikbare omgeving blijft zitten.

A.3.7. Planning

De totale duur van deze methodologie is gepland over een periode van 12 weken.

- Week 1-2: Rust aanleren
- Week 3-4: Sprint 1
- Week 5-6: Sprint 2
- Week 7-8: Sprint 3
- Week 9-10: Sprint 4
- Week 11-12: Sprint 5

A.4. Verwacht resultaat, conclusie

In dit onderzoek wordt er verwacht dat de ontwikkelde provisioner virtuele omgevingen van Windows en Linux kan opbouwen op een snellere en betrouwbaardere manier dan de huidige Vagrant provisioner. Dat de verkorte provisioning-tijd, minder mislukte runs en verlaagde manuele interventie er voor zorgt dat de virtuele omgeving betrouwbaarder opstart.

Voor de doelgroep, opleidingen die dat gebruik maken van provisioning, betekent dit dat lessen, oefeningen en projecten minder tijdverlies hebben door technische problemen. De bachelorproef zal niet alleen maar een proef-of-concept provisioner maken maar ook documentatie en een handleiding voor mogelijkheid van uitbreiding